

CSS

CHEAT SHEETS

A Complete Reference for Modern CSS

Selectors

Box Model

Typography

Colors

Flexbox

Grid

Positioning

Transitions

Animations

Transform

Responsive

Modern CSS

25 REFERENCE SHEETS

CONTENTS

FOUNDATIONS & SELECTORS

- 01 Architecture & Organization
- 02 Naming Conventions
- 03 Selectors: Basic & Combinators
- 04 Selectors: Pseudo-Classes
- 05 Selectors: Pseudo-Elements & Attribute
- 06 The Box Model
- 07 Borders & Outlines
- 08 Typography: Font Properties
- 09 Typography: Text Properties

BOX MODEL & TYPOGRAPHY

COLOR & BACKGROUNDS

- 10 Colors
- 11 Backgrounds
- 12 Gradients

LAYOUT

- 13 Flexbox: Container
- 14 Flexbox: Items
- 15 Grid: Container
- 16 Grid: Items & Placement
- 17 Positioning
- 18 Display & Overflow

MOTION & TRANSFORM

- 19 Transitions
- 20 Animations & Keyframes
- 21 Transform

RESPONSIVE DESIGN

- 22 Responsive: Media Queries
- 23 Responsive: Fluid Units & Functions

MODERN CSS

- 24 CSS Variables
- 25 Modern CSS

Sheet 1

Architecture & Organization

semantic grouping · file architecture · cascade order · specificity

Sheet 2

Naming Conventions

BEM · SMACSS · OOCSS · utility-first · custom properties

Sheet 3

Selectors: Basic & Combinators

element · .class · #id · * · A B · A > B · A + B · A ~ B

Sheet 4

Selectors: Pseudo-Classes

:hover · :focus · :nth-child · :not · :checked · :disabled

Sheet 5

Selectors: Pseudo-Elements & Attribute

::before · ::after · ::placeholder · [attr] · [attr=] · [attr^=]

Sheet 6

The Box Model

margin · padding · border · width · height · box-sizing

Sheet 7

Borders & Outlines

border-style · border-radius · outline · box-shadow · shorthand

Sheet 8

Typography: Font Properties

font-family · font-size · font-weight · font-style · @font-face

Sheet 9

Typography: Text Properties

text-align · text-decoration · text-transform · letter-spacing · white-space

Sheet 10

Colors

color · opacity · hex · rgb · hsl · rgba · hsla · currentColor

Sheet 11

Backgrounds

background-color · background-image · background-size · background-position · repeat

Sheet 12

Gradients

linear-gradient · radial-gradient · conic-gradient · stops · repeating

Sheet 13

Flexbox: Container

display:flex · flex-direction · flex-wrap · justify-content · align-items · gap

Sheet 14

Flexbox: Items

flex-grow · flex-shrink · flex-basis · flex · align-self · order

Sheet 15

Grid: Container

display:grid · grid-template-columns · fr · repeat · minmax · auto-fit

Sheet 16

Grid: Items & Placement

grid-column · grid-row · grid-area · span · grid-template-areas · named lines

Sheet 17

Positioning

position · top · right · bottom · left · z-index · stacking context

Sheet 18

Display & Overflow

display · overflow · visibility · float · clear · clip

Sheet 19

Transitions

transition-property · duration · timing-function · delay · shorthand

Sheet 20

Animations & Keyframes

@keyframes · animation-name · duration · iteration-count · fill-mode

Sheet 21

Transform

translate · rotate · scale · skew · matrix · transform-origin · 3D

Sheet 22

Responsive: Media Queries

@media · min-width · max-width · orientation · breakpoints · print

Sheet 23

Responsive: Fluid Units & Functions

rem · em · vh · vw · % · clamp() · min() · max() · calc()

Sheet 24

CSS Variables

--custom-properties · var() · fallback · :root · scope · override

Sheet 25

Modern CSS

:is() · :where() · :has() · @layer · @container · subgrid

DESCRIPTION

Well-organized CSS is written in a consistent order, grouped by concern, and split into purposeful layers. The four semantic groups — **Positioning**, **Box Model**, **Typography**, and **Visuals** — give every property a logical home inside every rule you write. Layering files as Base → Layout → Components → Utilities mirrors the natural cascade, keeping specificity low and overrides predictable. Consistent structure across a project means any developer can find, change, or debug a rule in seconds.

NOTES

- › Property order within a rule is a *convention*, not enforced by the browser — but enforceable with `stylelint-order`
- › The cascade reads files top-to-bottom; later rules with equal specificity win
- › `!important` breaks the cascade — avoid in components, reserve for utility overrides only
- › CSS `@layer` lets you declare layer order explicitly, making the architecture enforceable in-browser

SEMANTIC PROPERTY GROUPING — WRITE RULES IN THIS ORDER

GROUP	PROPERTIES
Positioning	position top right bottom left z-index float clear
Box Model	display flex grid width height min/max-width margin padding border box-sizing overflow
Typography	font-family font-size font-weight font-style line-height letter-spacing text-align text-transform text-decoration white-space
Visuals	color background opacity border-radius box-shadow outline cursor transition animation transform visibility

FILE ARCHITECTURE

```
css/  
  base/  
    _reset.css margins, box-sizing  
    _variables.css --tokens  
    _typography.css body, h1-h6  
  layout/  
    _grid.css page structure  
    _header.css  
    _footer.css  
  components/  
    _button.css  
    _card.css  
    _modal.css  
  utilities/  
    _spacing.css .mt-4, .p-2  
    _display.css .flex, .hidden  
  main.css imports all layers
```

CASCADE ORDER & @LAYER



```
/* main.css - declare layer order first, then import each file into its layer */  
@layer base, layout, components, utilities;
```

```
@import "base/_reset.css" layer(base);  
@import "base/_variables.css" layer(base);  
@import "layout/_grid.css" layer(layout);  
@import "components/_button.css" layer(components);  
@import "utilities/_spacing.css" layer(utilities);
```

SPECIFICITY REFERENCE

*		0,0,0
div, p, li		0,0,1
.class, [attr]		0,1,0
#id		1,0,0
style=""		inline
!important		wins all

COMMON MISTAKES

- › Writing properties in random order — makes rules hard to scan and diff
- › Using `#id` selectors in components — locks in high specificity, hard to override
- › Putting layout rules inside component files — breaks separation of concerns
- › Overusing `!important` to fix specificity wars instead of restructuring

TIPS & BEST PRACTICES

- › Use a linter (`stylelint` + `stylelint-order`) to enforce property order automatically
- › Keep specificity flat — prefer classes over chained element+class selectors
- › Use `@layer` in new projects — makes cascade order explicit and debuggable
- › One component = one file; name the file after the component (`_card.css`)

QUICK SUMMARY

- › Group properties: **Pos** → **Box** → **Type** → **Visual**
- › Layer files: Base → Layout → Components → Utilities
- › Later rules win ties; higher specificity always wins
- › `@layer` lets you control cascade order explicitly

DESCRIPTION

CSS naming conventions give classes meaning, prevent conflicts, and make code self-documenting. **BEM** (Block Element Modifier) is the most widely adopted: it encodes structure directly into class names. **SMACSS** categorizes rules by role and uses prefixes. **OOCSS** separates structure from skin. Utility-first (Tailwind-style) uses single-purpose classes. Custom property names follow a consistent `--category-variant` token pattern.

NOTES

- › BEM uses double-underscore for elements (`block__element`) and double-hyphen for modifiers (`block--modifier`)
- › SMACSS prefixes: `l-` for layout, `is-` for state — elements and modules are unprefixed
- › Utility classes trade HTML readability for zero CSS growth — every class does exactly one thing
- › Custom properties use `--category-variant` pattern: e.g. `--color-primary`, `--spacing-md`

EXAMPLE

```
.card { } /* BEM: Block */
.card__title { } /* BEM: Element */
.card--featured { } /* BEM: Modifier */

.l-grid { } /* SMACSS: Layout */
.is-active { } /* SMACSS: State */

.structure { width: 100%; } /* OOCSS: structure class */
.skin-dark { background: #111; } /* OOCSS: skin class */

.mt-4 { margin-top: 1rem; } /* Utility-first */
.flex { display: flex; }

:root {
  --color-primary: #5c6bc0; /* Custom property token */
  --spacing-md: 1rem;
}
```

CODE EXPLAINED

- `.card` — BEM block: the standalone, self-contained component
- `.card__title` — BEM element: a part that lives inside the block
- `.card--featured` — BEM modifier: a named variant of the block
- `.l-grid` — SMACSS layout prefix signals structural intent
- `.is-active` — SMACSS state prefix — toggled by JavaScript
- `.structure` / `.skin-*` — OOCSS separates layout from appearance
- `.mt-4` — utility class: one rule, one job, no exceptions
- `--color-primary` — token naming: category then variant

BEM PATTERN

SYNTAX	DESCRIPTION
<code>.block</code>	Standalone component (e.g. <code>.card</code>)
<code>.block__element</code>	Child part (e.g. <code>.card__title</code>)
<code>.block--modifier</code>	Variant (e.g. <code>.card--dark</code>)
<code>.block__el--mod</code>	Element with modifier

SMACSS PREFIXES

SYNTAX	DESCRIPTION
(none)	Base — element defaults, resets
<code>l-</code>	Layout — structural containers
(none)	Module — reusable UI components
<code>is-</code>	State — JS-toggled conditions
<code>theme-</code>	Theme — visual skin overrides

CUSTOM PROPERTY TOKENS

SYNTAX	DESCRIPTION
<code>--color-*</code>	e.g. <code>--color-primary</code>
<code>--spacing-*</code>	e.g. <code>--spacing-lg</code>
<code>--font-*</code>	e.g. <code>--font-body</code>
<code>--radius-*</code>	e.g. <code>--radius-sm</code>
<code>--shadow-*</code>	e.g. <code>--shadow-card</code>

⚠ COMMON MISTAKES

- › Mixing conventions in the same project — pick one and commit to it
- › Using vague names like `.box` or `.thing` — names should describe purpose
- › BEM modifiers that encode visuals: `.btn--red` — prefer semantic `.btn--danger`
- › Skipping the block on elements: `__title` instead of `.card__title`

✓ TIPS & BEST PRACTICES

- › BEM for component-based projects; utility-first when paired with Tailwind or similar
- › Use `is-` and `has-` prefixes even in BEM to signal JS-toggled state clearly
- › Name custom properties by role not value: `--color-danger` not `--color-red`
- › Document your convention in the project README so all contributors stay consistent

QUICK SUMMARY

- › BEM: `block__element--modifier` — the most widely used convention
- › SMACSS: categorize by role; prefix layout (`l-`) and state (`is-`)
- › OOCSS: separate structure (layout) from skin (appearance)
- › Custom properties: `--category-variant` token pattern throughout

DESCRIPTION

CSS selectors target the HTML elements you want to style. Basic selectors match by element type, class, ID, or all elements universally. Combinator selectors express structural relationships between elements — descendant, direct child, adjacent sibling, and general sibling. Every rule begins with a selector, and its specificity determines which rule wins when declarations conflict.

NOTES

- › Class selectors (`.class`) are the most reusable — prefer them over element or ID selectors in components
- › `#id` specificity is 1,0,0 — avoid in stylesheets, reserve for JS hooks or skip-nav links
- › The universal selector `*` matches every element — powerful in resets but avoid deep nesting for performance
- › Combinators add no specificity of their own beyond their individual selector parts

EXAMPLE

```
p          { color: #ccc; }           /* element */
.card      { padding: 1rem; }        /* class */
#hero      { height: 100vh; }        /* id */
*          { box-sizing: border-box; } /* universal */

/* Combinators */
nav a      { color: white; } /* descendant: any <a> inside <nav> */
ul > li    { list-style: none; } /* child: direct <li> of <ul> only */
h2 + p     { margin-top: 0; } /* adjacent: <p> immediately after <h2> */
h2 ~ p     { color: #999; } /* sibling: ALL <p> after <h2> in parent */

/* Compound */
a.active   { font-weight: 700; } /* element AND class */
li.item > a { color: #9c8fff; } /* direct child of classed element */
```

CODE EXPLAINED

- `p` — matches every `<p>` element on the page
- `.card` — matches every element with class `card`
- `#hero` — matches the single element with id `hero`
- `*` — universal: matches every element
- `nav a` — descendant: any `<a>` anywhere inside `<nav>`
- `ul > li` — child: only direct `<li>` children of `<ul>`
- `h2 + p` — adjacent: the `<p>` immediately after `<h2>`
- `h2 ~ p` — general sibling: all `<p>` after `<h2>` in the same parent

BASIC SELECTORS

SYNTAX	DESCRIPTION
<code>element</code>	Matches by tag: <code>p</code> , <code>h1</code> , <code>div</code>
<code>.class</code>	Matches elements with that class
<code>#id</code>	Matches the one element with that ID
<code>*</code>	Universal — matches every element
<code>el.class</code>	Element AND class — e.g. <code>a.active</code>
<code>.a.b</code>	Both classes present on one element

COMBINATORS

SYNTAX	DESCRIPTION
<code>A B</code>	Descendant — B anywhere inside A
<code>A > B</code>	Child — B is direct child of A
<code>A + B</code>	Adjacent — B immediately after A
<code>A ~ B</code>	Sibling — all B after A in same parent

SPECIFICITY VALUES

SYNTAX	DESCRIPTION
<code>*</code>	0,0,0 — no specificity
<code>element</code>	0,0,1 — one type
<code>.class</code>	0,1,0 — one class
<code>#id</code>	1,0,0 — one ID
<code>el.class</code>	0,1,1 — combined
<code>#id .class</code>	1,1,0 — combined

⚠ COMMON MISTAKES

- › Using `#id` in component CSS — creates specificity debt requiring `!important` to override
- › Over-qualifying: `div.card` instead of `.card` — reduces reusability without benefit
- › Confusing `A B` (any depth) with `A > B` (direct only) — descendants match at any depth
- › Long chains like `nav ul li a span` — brittle, breaks whenever HTML structure changes

✓ TIPS & BEST PRACTICES

- › Prefer class selectors for styling; element selectors only for base/reset rules
- › Use `>` child combinator to scope styles to direct children only — avoids deep inheritance leaks
- › Keep selector chains to 2–3 parts maximum for performance and maintainability
- › Use `:is()` to group selectors without repeating long combinator chains (see Sheet 25)

QUICK SUMMARY

- › `element · .class · #id · *` — the four basic selector types
- › `A B` descendant · `A > B` child · `A + B` adjacent · `A ~ B` sibling
- › Specificity: ID (1,0,0) > class (0,1,0) > element (0,0,1) > universal (0,0,0)
- › Keep selectors short and class-based for low-specificity, reusable styles

DESCRIPTION

Pseudo-classes select elements based on their *state* or *position* in the DOM — without adding any classes to HTML. Interaction pseudo-classes like `:hover` and `:focus` respond to user actions. Structural ones like `:nth-child()` target elements by their position among siblings. Form pseudo-classes like `:checked` and `:disabled` reflect input state in real time.

NOTES

- › Pseudo-classes have the same specificity as a class selector: 0,1,0
- › `:nth-child()` counts all sibling elements; `:nth-of-type()` counts only same-tag siblings
- › `:focus-visible` is preferred over `:focus` — shows rings for keyboard users only, not mouse clicks
- › `:not()` accepts a full selector list in modern CSS: `:not(.a, .b)`

EXAMPLE

```
a:hover           { color: #7986cb; }
input:focus       { outline: 2px solid #5c6bc0; }
input:focus-visible { outline: 2px solid #5c6bc0; } /* keyboard only */

li:first-child    { font-weight: 700; }
li:last-child     { border-bottom: none; }
li:nth-child(2)   { color: #ccc; }
li:nth-child(odd) { background: #1c2030; } /* every odd row */
li:nth-child(3n)  { color: #9c8fff; }      /* every 3rd */
li:nth-child(3n+1) { margin-top: 1rem; }    /* 1st of each group of 3 */

input:checked      { accent-color: #5c6bc0; }
input:disabled     { opacity: 0.4; cursor: not-allowed; }
.card:not(.featured) { opacity: 0.8; }
li:not(:last-child) { border-bottom: 1px solid #333; }
```

CODE EXPLAINED

- `:hover` — pointer device is over the element
- `:focus` — element has received focus (keyboard or click)
- `:focus-visible` — keyboard-driven focus only; skips mouse clicks
- `:first-child` / `:last-child` — positional within parent
- `:nth-child(n)` — n can be a number, keyword, or formula
- `odd` / `even` — shorthand for formulas `2n+1` / `2n`
- `3n+1` — matches 1, 4, 7, 10... (first of each group of 3)
- `:not(selector)` — excludes elements matching the argument

INTERACTION STATES

SYNTAX	DESCRIPTION
<code>:hover</code>	Pointer over element
<code>:focus</code>	Element is focused
<code>:focus-visible</code>	Keyboard focus only
<code>:active</code>	Being clicked / pressed
<code>:visited</code>	Link already visited
<code>:link</code>	Unvisited link

STRUCTURAL

SYNTAX	DESCRIPTION
<code>:first-child</code>	First among siblings
<code>:last-child</code>	Last among siblings
<code>:nth-child(n)</code>	By position, 1-based
<code>:nth-last-child(n)</code>	From the end
<code>:only-child</code>	Has no siblings
<code>:nth-of-type(n)</code>	By same-tag position

FORM & LOGIC

SYNTAX	DESCRIPTION
<code>:checked</code>	Checkbox/radio is on
<code>:disabled</code>	Input is disabled
<code>:required</code>	Input has required attr
<code>:valid</code> / <code>:invalid</code>	Validation state
<code>:placeholder-shown</code>	Placeholder is visible
<code>:not(sel)</code>	Excludes matching elements

⚠ COMMON MISTAKES

- › Styling `:focus` for visible rings — use `:focus-visible` to skip mouse-click rings
- › Assuming `:nth-child(2)` is the second of that tag type — it counts all siblings regardless of tag
- › Setting `outline: none` without an alternative — breaks keyboard navigation accessibility
- › Using `:hover` for critical interactions — hover does not exist on touch devices

✓ TIPS & BEST PRACTICES

- › Always style `:focus-visible` — never remove focus rings without a visible replacement
- › Use `:nth-child(odd/even)` for zebra-stripe tables instead of adding extra HTML classes
- › `li:not(:last-child) { border-bottom: 1px solid; }` — dividers without the final extra line
- › Chain pseudo-classes for compound states: `input:focus:not(:disabled)`

QUICK SUMMARY

- › Pseudo-classes select by state or position — no extra HTML markup needed
- › `:hover` / `:focus-visible` / `:active` — user interaction states
- › `:nth-child()` accepts numbers, `odd/even` keywords, and formulas like `2n+1`
- › `:not()` / `:checked` / `:disabled` — logic and form input states

Selectors: Pseudo-Elements & Attribute

`::before` · `::after` · `::placeholder` · `[attr]` · `[attr=]` · `[attr^=]`

DESCRIPTION

Pseudo-elements create or target sub-parts of an element that don't exist as separate HTML nodes. `::before` and `::after` insert generated content and require a content property — even an empty string — to render. Attribute selectors match elements by the presence or value of HTML attributes, enabling powerful targeting without extra classes — perfect for links, inputs, and icon systems.

NOTES

- Double colon (`::`) is the modern syntax — avoid single-colon for pseudo-elements in new code
- `::before` and `::after` are inline by default — set `display: block` or `position: absolute` as needed
- `content: ''` is required even for purely decorative pseudo-elements — without it nothing renders
- Attribute selectors have the same specificity as a class: 0,1,0

CODE EXPLAINED

- `::before` — generated content inserted before the element's content
- `::after` — generated content inserted after the element's content
- `content: ''` — required; can hold text, `url()`, or `counter()`
- `::placeholder` — styles input placeholder text
- `::selection` — styles user-highlighted text
- `[attr]` — element has the attribute, any value
- `[attr='val']` — exact match; `[attr^=]` starts; `[attr$=]` ends; `[attr*=]` contains
- `[lang]='en'` — exact `en` or starts with `en-`

GENERATED CONTENT

```
/* Generated content */
.card::before {
  content: '';           /* required - even empty */
  display: block;
  height: 4px;
  background: #5c6bc0; /* top accent */
}

.required::after {
  content: ' *';         /* literal text appended */
  color: #e57373;
}
```

TEXT SUB-PARTS & ATTRIBUTE SELECTORS

```
/* Targeting text sub-parts */
input::placeholder { color: #5a6285; font-style: italic; }
p::first-line      { font-weight: 700; }
p::selection        { background: #5c6bc0; color: white; }

/* Attribute selectors */
[disabled]          { opacity: 0.5; }           /* attribute exists */
[type="email"]       { border: 1px solid #ccc; } /* exact value */
[class^="icon-"]     { display: inline-block; } /* starts with */
[href$=".pdf"]        { color: #e57373; }        /* ends with */
[class*="col-"]      { float: left; }           /* contains */
[lang]="en"          { quotes: '""'; }          /* exact or en- prefix */
```

PSEUDO-ELEMENTS

SYNTAX	DESCRIPTION
<code>::before</code>	Generated content before
<code>::after</code>	Generated content after
<code>::placeholder</code>	Input placeholder text
<code>::first-line</code>	First rendered line of block
<code>::first-letter</code>	First character of block
<code>::selection</code>	User-highlighted text
<code>::marker</code>	List bullet or number

ATTRIBUTE SELECTORS

SYNTAX	DESCRIPTION
<code>[attr]</code>	Has attribute (any value)
<code>[attr='val']</code>	Exact value match
<code>[attr~='val']</code>	Value in space-separated list
<code>[attr^='val']</code>	Value starts with
<code>[attr\$='val']</code>	Value ends with
<code>[attr*='val']</code>	Value contains substring
<code>[attr]='val'</code>	Exact or starts with val-

CONTENT: VALUES

SYNTAX	DESCRIPTION
<code>''</code>	Empty string — decorative
<code>'text'</code>	Literal string content
<code>attr(href)</code>	Pull element attribute value
<code>counter(name)</code>	CSS counter value
<code>url('img.svg')</code>	Image as generated content
<code>none</code>	Suppresses the pseudo-element

△ COMMON MISTAKES

- Forgetting `content: ''` on `::before::after` — the element won't render at all
- Using `:before` (single colon) in new code — use double colon for all pseudo-elements
- Putting meaningful text in `::before::after` — generated content is invisible to screen readers
- Using `[attr*='val']` when you need exact match — substring match can hit unintended elements

✓ TIPS & BEST PRACTICES

- `::before::after` with `position: absolute` are perfect for decorative overlays without extra HTML
- Use `[href$='.pdf']` to auto-badge downloadable file links without adding classes
- Style `::placeholder` to match your design system — browsers show a faded fallback by default
- `::selection` adds brand character — match it to your primary accent color

QUICK SUMMARY

- `::before::after` insert generated content — always require a `content:` declaration
- Pseudo-elements target sub-parts of elements, not separate DOM nodes
- `[attr]` exists · `[attr='v']` exact · `[attr^=]` starts · `[attr$=]` ends · `[attr*=]` contains
- Both pseudo-elements and attribute selectors have class-level specificity (0,1,0)

DESCRIPTION

Every HTML element is a rectangular box composed of four layers from inside out: **content**, **padding**, **border**, and **margin**. By default (`box-sizing: content-box`), `width` applies only to the content area — padding and border expand the total size. Setting `box-sizing: border-box` includes padding and border inside the declared width, which is almost always what you want and is the standard in modern CSS resets.

EXAMPLE

```
/* Universal border-box reset — add to base/_reset.css */
*, ::before, ::after { box-sizing: border-box; }

.box {
  /* Sizing */
  width: 320px;
  height: 200px;

  /* Padding — space inside the border */
  padding: 1rem;           /* all sides */
  padding: 0.5rem 1rem;    /* vertical | horizontal */
  padding: 0.5rem 1rem 1.5rem 1rem; /* T R B L (clockwise) */

  /* Border */
  border: 2px solid #5c6bc0;

  /* Margin — space outside the border */
  margin: 0 auto;          /* center horizontally */
  margin-top: 2rem;
}

/* With border-box: declared width = total rendered width */
/* padding + border are INSIDE the 320px, not added on top */
```

NOTES

- › `* { box-sizing: border-box }` in your reset — makes element sizing intuitive project-wide
- › Margins can be negative; padding cannot go below zero
- › Vertical margins between adjacent block elements *collapse* to the larger of the two values — horizontal margins never collapse
- › `width: auto` fills available space minus margins; `width: 100%` fills the parent content box exactly

CODE EXPLAINED

- `box-sizing: border-box` — width/height include padding and border
- `width / height` — sets the box size (content only in content-box; total in border-box)
- `padding` — inner space between content and border
- `padding: 0.5rem 1rem` — shorthand: vertical | horizontal
- `padding: T R B L` — four-value clockwise shorthand
- `border` — drawn between padding and margin; takes space in content-box mode
- `margin` — outer space beyond the border
- `margin: 0 auto` — centers a block element horizontally

SHORTHAND PATTERNS

SYNTAX	DESCRIPTION
<code>margin: 1rem</code>	All four sides equal
<code>margin: 1rem 2rem</code>	Vertical Horizontal
<code>margin: 1rem 2rem 3rem</code>	Top LR Bottom
<code>margin: T R B L</code>	Each side clockwise

SIZING PROPERTIES

SYNTAX	DESCRIPTION
<code>width / height</code>	Dimension of content or box
<code>min-width / min-height</code>	Lower bound — never smaller
<code>max-width / max-height</code>	Upper bound — never larger
<code>width: auto</code>	Fill available space minus margins
<code>width: fit-content</code>	Shrink-wrap the content
<code>width: 100%</code>	100% of parent content width

OVERFLOW

SYNTAX	DESCRIPTION
<code>overflow: visible</code>	Default — content spills outside
<code>overflow: hidden</code>	Clips overflowing content
<code>overflow: scroll</code>	Always shows scrollbars
<code>overflow: auto</code>	Scrollbars only when needed
<code>overflow-x / overflow-y</code>	Control each axis separately

⚠ COMMON MISTAKES

- › Not using `box-sizing: border-box` — adding padding makes elements larger than their set width
- › Using `width: 100%` on elements with padding — in content-box mode the total exceeds 100%
- › Relying on margin collapse — it does not happen inside flex or grid containers, or with overflow set
- › Setting `margin: auto` on inline elements — centering only works on block-level boxes

✓ TIPS & BEST PRACTICES

- › Apply `box-sizing: border-box` universally via the `*` selector in your base reset
- › Use `max-width` instead of `width` for containers — allows shrinking on small screens
- › Padding for inner space, margin for outer space — they serve different purposes
- › `margin: 0 auto` with a defined `max-width` is the classic centered container pattern

QUICK SUMMARY

- › Layers inside-out: content → padding → border → margin
- › `box-sizing: border-box`: padding and border are inside the declared width
- › Padding is inside; margin is outside — neither can be negative (margin can, padding cannot)
- › Vertical block margins collapse; horizontal margins never do

DESCRIPTION

Borders are part of the box model and take up space in the layout. The `border` shorthand sets width, style, and color in one declaration — all three are required for a border to appear. `border-radius` rounds corners independently. `outline` sits outside the margin, takes no space, and is the browser's default focus indicator. `box-shadow` adds depth without affecting layout — it can be layered with comma-separated values.

NOTES

- › `border-style` is required — width and color alone produce no visible border
- › Outlines sit outside the margin and take no layout space — ideal for focus indicators that don't shift content
- › `box-shadow` comma-separated layers stack with the first listed on top
- › `border-radius: 50%` on a square element creates a perfect circle

EXAMPLE

```
/* Border shorthand: width | style | color */
.box { border: 2px solid #5c6bc0; }
.box { border-top: 1px dashed #ccc; }      /* single side */
.box { border-color: red green blue pink; } /* T R B L colors */

/* Border radius */
.card { border-radius: 8px; }                /* all corners */
.pill  { border-radius: 999px; }             /* pill / capsule */
.circle { border-radius: 50%; }              /* perfect circle */
.asymm { border-radius: 8px 0 8px 0; }      /* TL TR BR BL */
.fancy { border-radius: 2rem 0.5rem; }

/* Outline — sits outside the border, no layout impact */
:focus-visible {
  outline: 2px solid #5c6bc0;
  outline-offset: 3px;                      /* gap between element and ring */
}

/* box-shadow: offset-x offset-y blur spread color */
.card { box-shadow: 0 4px 12px rgba(0,0,0,0.3); }
.deep { box-shadow: 0 2px 4px rgba(0,0,0,0.2),
                  0 8px 24px rgba(0,0,0,0.15); } /* layered */
.inner { box-shadow: inset 0 2px 4px rgba(0,0,0,0.3); }
```

CODE EXPLAINED

- `border: 2px solid #5c6bc0` — shorthand: width style color
- `border-top` — targets a single side; any side can be set independently
- `border-radius: 8px` — rounds all four corners equally
- `border-radius: 50%` — circle when element width equals height
- `outline` — like border but outside the box model, takes no space
- `outline-offset` — gap between the element edge and the outline
- `box-shadow: x y blur spread color` — all space-separated
- `inset` keyword — shadow drawn inside the element boundaries

BORDER STYLES

SYNTAX	DESCRIPTION
<code>solid</code>	Continuous line
<code>dashed</code>	Dash pattern
<code>dotted</code>	Dot pattern
<code>double</code>	Two parallel lines
<code>groove / ridge</code>	3D carved/raised effect
<code>none / hidden</code>	No border rendered

BORDER-RADIUS PATTERNS

SYNTAX	DESCRIPTION
<code>8px</code>	All corners equal
<code>8px 0</code>	TL+BR TR+BL
<code>TL TR BR BL</code>	Four corners individually
<code>50%</code>	Circle on a square element
<code>999px</code>	Pill / capsule shape
<code>8px / 4px</code>	Horizontal / vertical radii

BOX-SHADOW VALUES

SYNTAX	DESCRIPTION
<code>offset-x</code>	Horizontal shadow shift
<code>offset-y</code>	Vertical shadow shift
<code>blur-radius</code>	Edge softness
<code>spread-radius</code>	Expand or contract size
<code>color</code>	Shadow color — use <code>rgba</code>
<code>inset</code>	Inner shadow keyword

⚠ COMMON MISTAKES

- › Setting `border-width` and `border-color` without `border-style` — border won't appear
- › Using `outline: none` without an alternative — breaks keyboard accessibility
- › Expecting `border-radius` to clip child content — add `overflow: hidden` to the parent too
- › Using `box-shadow spread` as a focus ring — outline is more appropriate and accessible

✓ TIPS & BEST PRACTICES

- › Use `outline-offset` to create a visible gap for a polished focus ring without content shift
- › Layer multiple `box-shadow` values comma-separated for realistic depth and ambient light
- › Use `border: none` rather than `border: 0` — clearer intent
- › For a one-sided accent, set only `border-top` or `border-left` rather than overriding all four

QUICK SUMMARY

- › `border` shorthand: width | style | color — all three required to render
- › `border-radius: 50%` = circle · `999px` = pill
- › Outlines take no layout space — never remove focus outlines without a visible replacement
- › `box-shadow: x y blur spread color` · use `inset` for inner shadows

DESCRIPTION

Font properties control the typeface, size, weight, and style of text. `font-family` accepts a comma-separated fallback stack ending with a generic family. `font-size` is the foundation for relative units — `rem` scales with the root, `em` with the parent. The `font` shorthand combines family, size, weight, style, and line-height in one declaration. `@font-face` loads custom fonts from files and should always include `font-display: swap` to avoid invisible text during load.

EXAMPLE

```
/* Family stack — first available font wins */
body { font-family: 'DM Sans', Helvetica, Arial, sans-serif; }
code { font-family: 'Source Code Pro', Consolas, monospace; }

/* Size and weight */
h1 { font-size: 2.5rem; font-weight: 700; }
h2 { font-size: 1.75rem; font-weight: 600; }
p { font-size: 1rem; font-weight: 400; }
small { font-size: 0.875rem; }

/* Style and variant */
em { font-style: italic; }
.caps { font-variant: small-caps; }

/* Font shorthand: style weight size/line-height family */
p { font: 400 1rem/1.6 'DM Sans', sans-serif; }

/* @font-face — self-hosted custom font */
@font-face {
  font-family: 'MyFont';
  src: url('../fonts/myfont.woff2') format('woff2'),
       url('../fonts/myfont.woff') format('woff');
  font-weight: 400 700; /* weight range this file covers */
  font-style: normal;
  font-display: swap; /* show fallback instantly */
}
```

NOTES

- › Always end a `font-family` stack with a generic: `serif`, `sans-serif`, `monospace`, `cursive`, or `fantasy`
- › `font-weight` accepts keywords (`bold`) or numeric 100–900 in increments of 100
- › Prefer `rem` over `px` for font-size — respects the user's browser font-size preference
- › Variable fonts support fractional weights and custom axes via `font-variation-settings`

CODE EXPLAINED

- `font-family` — comma list; first available wins; end with a generic
- `font-size: 2.5rem` — 2.5× the root font-size (typically 40px)
- `font-weight: 700` — bold; 400 = normal, 900 = black
- `font-style: italic` — uses the italic variant of the font if available
- `font: 400 1rem/1.6 ...` — shorthand: weight size/line-height family
- `@font-face` — declares a custom font and its source file(s)
- `font-weight: 400 700` — tells the browser this file covers that weight range
- `font-display: swap` — render the fallback immediately; swap when font loads

GENERIC FONT FAMILIES

SYNTAX	DESCRIPTION
<code>serif</code>	Georgia, Times — has serifs
<code>sans-serif</code>	Helvetica, Arial — no serifs
<code>monospace</code>	Courier, Consolas — fixed-width
<code>cursive</code>	Handwriting-style
<code>fantasy</code>	Decorative / display
<code>system-ui</code>	OS default UI font

FONT-WEIGHT VALUES

SYNTAX	DESCRIPTION
<code>100</code>	Thin
<code>300</code>	Light
<code>400 / normal</code>	Regular (default)
<code>500</code>	Medium
<code>600</code>	Semi-bold
<code>700 / bold</code>	Bold
<code>900</code>	Black / Heavy

FONT-SIZE UNITS

SYNTAX	DESCRIPTION
<code>px</code>	Absolute pixels — avoid for body text
<code>rem</code>	Root-relative — preferred
<code>em</code>	Parent-relative — compounds in nesting
<code>%</code>	Percentage of parent font-size
<code>vw</code>	Viewport-width relative
<code>clamp(min, vw, max)</code>	Fluid — scales with viewport

⚠ COMMON MISTAKES

- › Omitting the generic fallback in `font-family` — no safety net if the named font fails to load
- › Using `px` for body `font-size` — prevents users from scaling text in browser settings
- › Setting `font-weight: bold` when the font only has 400/700 — browser fake-bolds look poor
- › Not adding `font-display: swap` in `@font-face` — causes FOIT (invisible text during font load)

✓ TIPS & BEST PRACTICES

- › Set `font-size: 62.5%` on `:root` so `1rem = 10px` for easier mental math throughout the project
- › Use Google Fonts or Bunny Fonts [<link>](#) for quick projects instead of self-hosted `@font-face`
- › Pair fonts in 2s: one distinctive display font for headings, one readable sans-serif for body
- › Use `clamp(1rem, 2.5vw, 2rem)` for fluid type that scales with the viewport without breakpoints

QUICK SUMMARY

- › `font-family` — always end the stack with a generic fallback
- › Prefer `rem` for font-size — scales with the user's browser preference
- › `font-weight: 100 thin → 400 normal → 700 bold → 900 black`
- › `@font-face` loads custom fonts; always use `font-display: swap`

DESCRIPTION

Text properties control how characters and lines are arranged, decorated, and spaced independently of the font itself. `text-align` sets horizontal alignment within a block. `line-height` and `letter-spacing` tune readability and visual rhythm. The classic truncation pattern — `white-space: nowrap + overflow: hidden + text-overflow: ellipsis` — clips overflowing text to a single line with an ellipsis.

NOTES

- Unitless `line-height` (e.g. `1.6`) is preferred — it scales with the element's own font-size
- `text-align: justify` creates uneven word spacing — use sparingly and only for long-form prose
- `white-space: nowrap` prevents wrapping; `pre` preserves all whitespace as written
- `text-decoration` is a shorthand accepting line type, style, color, and thickness

CODE EXPLAINED

- `text-align` — left (default LTR), center, right, justify
- `text-decoration: none` — removes underline from links
- `underline wavy #e57373` — style and color in the shorthand
- `text-transform: uppercase` — CSS transform, HTML content unchanged
- `letter-spacing: -0.03em` — negative tightens; positive loosens
- `line-height: 1.6` — unitless, relative to the element's own font-size
- `white-space: nowrap` — prevents line breaks inside the element
- `text-overflow: ellipsis` — shows ... when clipped (needs `overflow: hidden + nowrap`)

EXAMPLE

```
/* Alignment */
.left { text-align: left; } /* default for LTR */
.center { text-align: center; }
.right { text-align: right; }
.justify { text-align: justify; }

/* Decoration */
a { text-decoration: none; }
.strike { text-decoration: line-through; }
.underline { text-decoration: underline; }
.fancy { text-decoration: underline wavy #e57373; }
.thick-under { text-decoration: underline 2px; }

/* Transform — applied via CSS, HTML unchanged */
.upper { text-transform: uppercase; }
.lower { text-transform: lowercase; }
.caps { text-transform: capitalize; }

/* Spacing */
h1 { letter-spacing: -0.03em; } /* tight heading */
.label { letter-spacing: 0.12em; } /* airy uppercase label */
p { line-height: 1.6; } /* unitless preferred */
p { word-spacing: 0.05em; }

/* Truncation recipe — all three required */
.truncate {
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis; /* shows ... at clip point */
}
```

TEXT-ALIGN VALUES

SYNTAX	DESCRIPTION
left	Default for LTR text
right	Right-aligned
center	Centered
justify	Fills line width
start / end	Logical — respects writing direction

TEXT-DECORATION VALUES

SYNTAX	DESCRIPTION
none	Removes all decoration
underline	Line below text
overline	Line above text
line-through	Strikethrough
underline dotted	Dotted underline style
underline wavy red	Wavy colored underline

WHITE-SPACE VALUES

SYNTAX	DESCRIPTION
normal	Collapse spaces, wrap (default)
nowrap	No line breaks
pre	Preserve whitespace, no wrap
pre-wrap	Preserve whitespace, allow wrap
pre-line	Collapse spaces, preserve newlines
break-spaces	Like pre-wrap, spaces wrap

COMMON MISTAKES

- Using `text-transform` in HTML attributes — if design changes, markup must change too; use CSS
- Setting `text-overflow: ellipsis` alone — requires `overflow: hidden + white-space: nowrap`
- Using `line-height: 1.6px` with a unit — unitless scales with font-size; px does not
- Applying `letter-spacing` to body text — best reserved for headings and uppercase UI labels

TIPS & BEST PRACTICES

- Use unitless `line-height`: 1.5–1.7 for body, 1.1–1.2 for headings
- Negative `letter-spacing` (–0.02em to –0.04em) on large headings looks more intentional and refined
- Memorise the truncation recipe — it's one of the most commonly needed CSS patterns
- Use `text-wrap: balance` (modern) on headings to avoid awkward single-word last lines

QUICK SUMMARY

- `text-align`: left | center | right | justify | start | end
- `text-decoration`: none | underline | line-through — accepts style and color
- Truncation: `white-space: nowrap + overflow: hidden + text-overflow: ellipsis`
- Use unitless `line-height` — it scales relative to the element's own font-size

DESCRIPTION

CSS colors can be written in several formats: named keywords, hexadecimal, `rgb()`, and `hsl()`. The `hsl()` format — hue, saturation, lightness — is the most human-readable for building and adjusting palettes. Modern CSS unifies `rgb/rgba` and `hsl/hsla` into single functions with an optional slash-separated alpha. `opacity` affects the entire element including its children; an alpha channel in a color value affects only that single property.

EXAMPLE

```
/* Named keyword */
p { color: tomato; }

/* Hex: #RRGGBB — or #RGB shorthand */
h1 { color: #5c6bc0; }
h1 { color: #59c; }           /* expands to #5599cc */

/* RGB — legacy and modern */
.a { color: rgb(92, 107, 192); }
.b { color: rgb(92 107 192 / 0.8); } /* modern + alpha */

/* HSL — hue 0–360, saturation %, lightness % */
.c { color: hsl(231, 44%, 56%); }
.d { color: hsl(231 44% 56% / 0.5); } /* modern + alpha */

/* opacity vs alpha channel */
.whole { opacity: 0.5; }           /* element + ALL children */
.precise { color: rgb(0 0 0 / 0.5); } /* only this color property */

/* currentColor — inherits computed color value */
.icon svg { fill: currentColor; }
.btn      { border: 2px solid currentColor; }
```

NOTES

- Modern syntax uses spaces and a slash for alpha: `rgb(0 0 0 / 0.5)` and `hsl(240 100% 50% / 0.5)`
- `opacity: 0.5` makes the element and all its children semi-transparent; color alpha only affects that property
- `currentColor` inherits the element's computed `color` value — ideal for SVG fills and matching borders
- CSS Color Level 4 adds `oklch()` for perceptually uniform color spaces that support wide-gamut displays

CODE EXPLAINED

- `color: tomato` — named keyword; ~140 named colors exist
- `#5c6bc0` — hex: two digits per channel (RR GG BB) in base-16
- `#59c` — three-digit shorthand: each digit is doubled
- `rgb(92 107 192 / 0.8)` — modern syntax: channels then alpha after /
- `hsl(231 44% 56%)` — hue 0–360°, saturation %, lightness %
- `opacity: 0.5` — entire element and all descendants at 50%
- `rgb(0 0 0 / 0.5)` — only that color declaration is semi-transparent
- `currentColor` — inherits the element's computed `color` value

COLOR FORMATS

SYNTAX	DESCRIPTION
keyword	Named: red, tomato, transparent
#RRGGBB	Hex 6-digit
#RGB	Hex 3-digit shorthand
#RRGGBBAA	Hex with alpha
rgb(R G B)	0–255 per channel
rgb(R G B / A)	With alpha 0–1
hsl(H S% L%)	Hue 0–360, sat %, light %
hsl(H S% L% / A)	HSL with alpha

HSL HUE REFERENCE

SYNTAX	DESCRIPTION
0° / 360°	Red
30°	Orange
60°	Yellow
120°	Green
180°	Cyan
240°	Blue
270°	Purple
300°	Magenta

SPECIAL VALUES

SYNTAX	DESCRIPTION
transparent	rgb(0 0 0 / 0) — fully clear
currentColor	Inherits element's color
inherit	Force inheritance from parent
initial	Reset to browser default
oklch(L C H)	Perceptually uniform (L4)

COMMON MISTAKES

- Using `opacity: 0.5` when only the background should be transparent — text fades too
- Writing raw hex values without custom properties — impossible to update consistently
- Using old comma syntax `hsl(H, S%, L%)` in new code — prefer the modern space syntax
- Assuming `#000` and `#000000` differ — they are identical

TIPS & BEST PRACTICES

- Use `hsl()` for palettes — adjust lightness to create tints (80%) and shades (30%) of one hue
- Store all colours as custom properties: `--color-primary: hsl(231 44% 56%)`
- Use `currentColor` for SVG fills and borders that must always match the text colour
- Target WCAG AA contrast: at least 4.5:1 for normal text, 3:1 for large text

QUICK SUMMARY

- Formats: keyword · `#hex` · `rgb()` · `hsl()` — all accept an optional alpha
- HSL is most intuitive for palette building — tweak lightness to make tints and shades
- `opacity` affects the whole element; color alpha affects only that one property
- `currentColor` inherits the element's `color` — great for SVG and border consistency

DESCRIPTION

The `background` shorthand paints a visual layer behind an element's content and padding. Multiple background layers can be stacked with comma-separated values — the first listed is on top. `background-size: cover` fills the container and may crop the image; `contain` fits the whole image inside without cropping but may leave gaps. `background-color` is always the bottom-most layer and cannot be layered in the image stack.

EXAMPLE

```
/* Solid color */
.box { background-color: #1c2030; }
.box { background: #1c2030; }           /* shorthand equivalent */

/* Image */
.hero {
  background-image: url('../img/hero.jpg');
  background-size:   cover;
  background-position: center;
  background-repeat: no-repeat;
}

/* Full shorthand: color image position/size repeat attachment */
.hero { background: #1c2030 url('../img/hero.jpg') center/cover no-repeat; }

/* Multiple layers – gradient overlay on top of image */
.hero {
  background:
    linear-gradient(rgba(0,0,0,0.55), rgba(0,0,0,0.55)),
    url('../img/hero.jpg') center/cover no-repeat;
}

/* Gradient text */
.grad-text {
  background: linear-gradient(135deg, #9c8fff, #64b5f6);
  -webkit-background-clip: text;
  background-clip: text;
  color: transparent;
}
```

NOTES

- Multiple backgrounds: comma-separated — first value is the topmost layer
- `background-color` sits below all image layers; it cannot appear in the comma-separated list
- `background-attachment: fixed` creates parallax but causes performance issues on mobile
- `background-clip: text` + `color: transparent` creates gradient text effects

CODE EXPLAINED

- `background-color` — solid fill; always the bottom-most layer
- `background-image: url()` — path to an image file or a gradient function
- `background-size: cover` — fills the element; may crop image edges
- `background-size: contain` — shows full image; may leave gaps
- `background-position: center` — places the focal point of the image
- `background-repeat: no-repeat` — prevents default tiling behaviour
- Multiple layers comma-separated — first listed renders on top
- `background-clip: text` + `color: transparent` = gradient text

BACKGROUND-SIZE VALUES

SYNTAX	DESCRIPTION
<code>auto</code>	Natural image size (default)
<code>cover</code>	Fill — may crop
<code>contain</code>	Fit entirely — may gap
<code>100% 100%</code>	Stretch to fill exactly
<code>50%</code>	Half the element width
<code>200px 100px</code>	Explicit W and H

⚠ COMMON MISTAKES

- Forgetting `no-repeat` on a hero image — it tiles by default
- Using the `background` shorthand without specifying all needed values — omitted sub-properties reset to defaults
- Using `background-attachment: fixed` on mobile — often disabled by mobile browsers for performance
- Using background images for meaningful content — use `<img alt='...'>` for accessibility

BACKGROUND-POSITION VALUES

SYNTAX	DESCRIPTION
<code>center</code>	Centre on both axes
<code>top / bottom</code>	Top or bottom edge
<code>left / right</code>	Left or right edge
<code>50% 25%</code>	Percentage x y
<code>20px 40px</code>	Offset from top-left
<code>right 20px bottom 10px</code>	Edge-relative offset

✓ TIPS & BEST PRACTICES

- Use the multi-layer technique to add a dark overlay on hero images without an extra HTML element
- `background: center/cover no-repeat` is the go-to pattern for any full-bleed image
- Always provide a `background-color` fallback when using image backgrounds
- Gradient text: pair `background-clip: text` with `color: transparent`

BACKGROUND-REPEAT VALUES

SYNTAX	DESCRIPTION
<code>repeat</code>	Default — tile both axes
<code>no-repeat</code>	No tiling
<code>repeat-x</code>	Tile horizontal only
<code>repeat-y</code>	Tile vertical only
<code>space</code>	Tile with equal spacing
<code>round</code>	Scale to fit whole tiles

QUICK SUMMARY

- `cover` fills and may crop · `contain` fits without cropping
- Shorthand: color image position/size repeat — unspecified values reset to defaults
- Multiple layers comma-separated — first listed is on top
- Always pair image backgrounds with `no-repeat` and a colour fallback

EXAMPLE

```
/* Linear */
.a { background: linear-gradient(to right, #5c6bc0, #9c8fff); }
.b { background: linear-gradient(135deg, #5c6bc0 0%, #9c8fff 100%); }

/* Multi-stop */
.rainbow {
  background: linear-gradient(
    to right, #e57373, #ffb74d, #81c784, #64b5f6, #9c8fff
  );
}

/* Hard stop — no blend at 50% */
.stripe { background: linear-gradient(to right, #5c6bc0 50%, #1c2030 50%); }

/* Radial */
.glow {
  background: radial-gradient(circle at center, #9c8fff 0%, #0f1117 70%);
}

/* Conic — pie chart slices */
.pie {
  background: conic-gradient(
    #e57373 0% 33%, #81c784 33% 66%, #64b5f6 66% 100%
  );
}

/* Repeating stripes */
.stripes {
  background: repeating-linear-gradient(
    45deg,
    #5c6bc0 0px, #5c6bc0 10px,
    #1c2030 10px, #1c2030 20px
  );
}
```

DESCRIPTION

CSS gradients are `<image>` values used anywhere `background-image` is accepted. `linear-gradient()` transitions colours along a straight line; `radial-gradient()` radiates from a centre point; `conic-gradient()` sweeps around a point like a pie chart. Colour stops define where each colour starts and ends — explicit positions give precise control. Hard stops — two colours at the same position — create sharp edges with zero blending.

NOTES

- Gradients are images — they work in `background`, `border-image`, and `mask`, not in `color`
- Without explicit stop positions, colours are evenly distributed along the gradient axis
- Hard stops (same position twice) create crisp colour changes — great for stripes and sections
- Gradients are resolution-independent — they render sharp at any size unlike raster images

CODE EXPLAINED

- `to right` — direction keyword; degrees also work (`135deg`)
- `0%, 100%` — explicit stop positions along the gradient axis
- Hard stop: same position twice — instant colour change, no blend
- `radial-gradient(circle at center)` — circular radiate from a point
- `conic-gradient` — sweeps around a centre point; ideal for pie charts
- `red 20% 40%` — colour holds solid from 20% to 40%
- `repeating-linear-gradient` — tiles the defined pattern endlessly
- `transparent` as a stop fades the gradient to nothing

LINEAR-GRADIENT DIRECTION		COLOUR STOP SYNTAX		GRADIENT TYPES	
SYNTAX	DESCRIPTION	SYNTAX	DESCRIPTION	SYNTAX	DESCRIPTION
to right	Left → Right	red	Auto-positioned stop	linear-gradient()	Straight-line transition
to bottom	Top → Bottom (default)	red 20%	Stop at 20% of axis	radial-gradient()	Outward from centre
to bottom right	Diagonal	red 20% 40%	Holds from 20%–40%	conic-gradient()	Sweeps around a point
45deg	45-degree angle	red 50%, blue 50%	Hard stop — no blend	repeating-linear-gradient()	Repeating linear
135deg	135-degree angle	transparent	Fade to nothing	repeating-radial-gradient()	Repeating rings
–45deg	Negative angle	20px	Absolute position	repeating-conic-gradient()	Repeating slices

⚠ COMMON MISTAKES

- Using a gradient as a `color` value — gradients are images, not colours
- Omitting explicit stop positions — browser distribution may not match your intent
- Forgetting `-webkit-background-clip: text` alongside `background-clip: text` for Safari
- Repeating gradients where stops don't cleanly tile — causes visible seam at the repeat boundary

✓ TIPS & BEST PRACTICES

- `linear-gradient(to bottom, color, transparent)` — classic fade-out overlay on images
- Hard stops create crisp CSS stripes — useful for dividers, progress bars, and colour blocks
- Conic gradients make pure-CSS pie charts — set stop percentages to slice proportions
- Store reusable gradients as custom properties: `--gradient-brand: linear-gradient(...)`

QUICK SUMMARY

- `linear-gradient(dir, stops) · radial-gradient(shape, stops) · conic-gradient(stops)`
- Gradients are images — use wherever `background-image` is accepted
- Hard stop: same position twice — instant colour change with zero blending
- `repeating-*gradient()` tiles the defined stop pattern endlessly

DESCRIPTION

Flexbox is a one-dimensional layout system that arranges direct children along a single axis — row or column. `display: flex` on a container makes its children flex items. The **main axis** is defined by `flex-direction`; the **cross axis** is perpendicular. `justify-content` distributes items along the main axis; `align-items` aligns them on the cross axis. `gap` replaces old margin hacks for consistent spacing between items.

EXAMPLE

```
.container {
  display: flex;
  flex-direction: row; /* row | column | row-reverse | column-reverse */
  flex-wrap: wrap;     /* nowrap | wrap | wrap-reverse */
  gap: 1rem;           /* uniform gap between all items */
  gap: 1rem 2rem;      /* row-gap | column-gap */
}

/* Main-axis alignment */
.a { justify-content: flex-start; } /* default */

/* Cross-axis alignment */
.e { align-items: stretch; }      /* default */

/* Classic centering pattern */
.center {
  display: flex;
  justify-content: center;
  align-items: center;
}

/* Nav: logo left, links right */
.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

NOTES

- Flexbox is one-dimensional — use Grid when you need both rows and columns simultaneously
- `gap` adds space between items only, not before the first or after the last
- `flex-wrap: wrap` lets items flow onto new lines; without it items shrink to stay on one line
- `align-content` aligns multiple wrapped lines; `align-items` aligns within each line

CODE EXPLAINED

- `display: flex` — block-level flex container; children become flex items
- `flex-direction: row` — main axis is horizontal (default)
- `flex-direction: column` — main axis is vertical
- `flex-wrap: wrap` — items wrap onto new lines when they overflow the container
- `gap: 1rem` — uniform spacing between all items with no edge space
- `justify-content` — alignment along the main axis
- `align-items: center` — centres items on the cross axis
- `justify-content: center + align-items: center` — centres anything

JUSTIFY-CONTENT

SYNTAX	DESCRIPTION
<code>flex-start</code>	Pack to start (default)
<code>flex-end</code>	Pack to end
<code>center</code>	Centre along main axis
<code>space-between</code>	Equal space between items
<code>space-around</code>	Equal space around items
<code>space-evenly</code>	Equal space everywhere

ALIGN-ITEMS

SYNTAX	DESCRIPTION
<code>stretch</code>	Fill cross axis (default)
<code>flex-start</code>	Align to cross-start
<code>flex-end</code>	Align to cross-end
<code>center</code>	Centre on cross axis
<code>baseline</code>	Align text baselines

FLEX-DIRECTION

SYNTAX	DESCRIPTION
<code>row</code>	L → R, horizontal (default)
<code>row-reverse</code>	R → L, horizontal
<code>column</code>	T → B, vertical
<code>column-reverse</code>	B → T, vertical

COMMON MISTAKES

- Using margins instead of `gap` — margins add unwanted space before first and after last item
- Applying `align-items` expecting multi-line behaviour — `align-content` handles multiple lines
- Assuming `justify-content` is always horizontal — it follows the main axis, which changes with `flex-direction`
- Setting flex properties on items instead of the container — container properties must go on the parent

TIPS & BEST PRACTICES

- `display:flex; justify-content:center; align-items:center` — the universal one-liner centering pattern
- `margin-left: auto` on a flex item pushes it and everything after it to the far end of the main axis
- Prefer `gap` over margins between items — no edge-case spacing and works in both directions
- `flex-direction: column` on the body or app wrapper creates a sticky-footer layout

QUICK SUMMARY

- `display: flex` on the container; direct children become flex items
- `flex-direction` sets main axis; `justify-content` distributes along it
- `align-items` aligns on cross axis; `align-content` aligns multiple wrapped lines
- `gap` is the clean modern way to space items — no margin hacks needed

DESCRIPTION

Flex item properties are placed on the children of a flex container, not the container itself. `flex-grow` distributes extra space proportionally; `flex-shrink` controls compression when space is tight; `flex-basis` sets the initial size before growing or shrinking. The flex shorthand combines all three. `align-self` overrides the container's `align-items` for a single item. `order` changes visual position without altering DOM order.

EXAMPLE

```
/* These properties go on the ITEM, not the container */
.item {
  flex-grow: 1;    /* share of extra space to absorb */
  flex-shrink: 1;  /* share to give up when tight */
  flex-basis: 0;   /* starting size before grow/shrink */
}

/* flex shorthand: grow | shrink | basis */
.item { flex: 1; }           /* 1 1 0 - free grow/shrink */
.item { flex: auto; }        /* 1 1 auto - from natural size */
.item { flex: none; }        /* 0 0 auto - rigid */
.item { flex: 0 1 200px; }   /* max 200px, can shrink, no grow */

/* Classic sidebar + main layout */
.sidebar { flex: 0 0 260px; } /* fixed 260px */
.main    { flex: 1; }        /* fills the rest */

/* align-self overrides container align-items for one item */
.special { align-self: flex-end; }
.tall    { align-self: stretch; }

/* order - default 0; lower numbers appear first */
.lead { order: -1; }
.last { order: 1; }
```

NOTES

- flex: 1 expands to flex: 1 1 0 — item grows and shrinks from a zero starting size
- flex: auto = flex: 1 1 auto — grows and shrinks from the item's natural content size
- flex: none = flex: 0 0 auto — completely rigid; ignores available or insufficient space
- order is visual only — DOM order, tab focus, and screen readers are unaffected

CODE EXPLAINED

- flex-grow: 1 — absorbs 1 share of leftover space after fixed items are placed
- flex-shrink: 1 — gives up 1 share when there is not enough room
- flex-basis: 0 — starts from zero; all space is distributed via grow
- flex: 1 — shorthand for grow=1, shrink=1, basis=0
- flex: none — rigid item; never grows or shrinks
- flex: 0 0 260px — fixed 260px sidebar that will not flex
- align-self — overrides align-items for this one item only
- order: -1 — appears before all order:0 items visually

FLEX SHORTHAND

SYNTAX	DESCRIPTION
flex: 1	1 1 0 — free growth from zero
flex: auto	1 1 auto — grow from natural size
flex: none	0 0 auto — completely rigid
flex: 0 1 auto	Default — shrinks, no grow
flex: 2	Takes 2× share vs flex:1 items
flex: 0 0 300px	Fixed 300px, no flexibility

ALIGN-SELF VALUES

SYNTAX	DESCRIPTION
auto	Inherit container align-items
flex-start	Align to cross-start
flex-end	Align to cross-end
center	Centre on cross axis
stretch	Fill cross axis height
baseline	Align text baselines

COMMON PATTERNS

SYNTAX	DESCRIPTION
flex: 1 on all items	Equal-width columns
flex: 0 0 Npx	Fixed sidebar
flex: 1 on one item	Fills remaining space
margin-left: auto	Push item to far end
order: -1	Move item to visual front

COMMON MISTAKES

- Confusing flex: 1 (basis=0) with flex: auto (basis=auto) — they distribute space differently when items have varying content
- Using order to reorder meaningful content — DOM order still controls focus and accessibility
- Setting flex-grow without resetting flex-basis — use the shorthand flex: 1 to set all three correctly
- Applying flex item properties to the container element instead of the child items

TIPS & BEST PRACTICES

- flex: 1 on content + flex: 0 0 Npx on sidebar = robust two-column layout
- margin-left: auto on a flex item absorbs all available space — great for pushing nav items right
- Different flex-grow values create proportional layouts: 2 vs 1 gives a 2:1 ratio
- Always use the flex shorthand rather than individual properties — it correctly resets all three

QUICK SUMMARY

- flex: grow shrink basis — the item's complete sizing recipe
- flex: 1 = freely growing · flex: none = rigid · flex: auto = from natural size
- align-self overrides the container's align-items for a single item only
- order changes visual order only — DOM and accessibility order are unchanged

EXAMPLE

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;      /* 3 equal columns */
  grid-template-columns: repeat(3, 1fr);    /* same thing */
  grid-template-rows: auto;               /* rows size to content */
  gap: 1rem;                             /* row and column gap */
  gap: 1rem 2rem;                         /* row-gap | column-gap */
}

/* Mixed track sizes */
.layout {
  grid-template-columns: 260px 1fr;        /* fixed sidebar + fluid main */
  grid-template-columns: 1fr 2fr 1fr;     /* 1:2:1 ratio */
}

/* Responsive — no media queries needed */
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
}

/* Named template areas */
.page {
  display: grid;
  grid-template-columns: 260px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    'header header'
    'sidebar main'
    'footer footer';
}

/* Implicit rows — controls height of auto-created rows */
.feed { grid-auto-rows: 200px; }
```

DESCRIPTION

CSS Grid is a two-dimensional layout system — it controls rows and columns simultaneously, `display: grid` on a container makes its direct children grid items. Tracks are defined with `grid-template-columns` and `grid-template-rows`. The `fr` unit distributes free space proportionally. `repeat()` reduces repetition; `minmax()` sets a size range. Combining `auto-fit` with `minmax()` creates fully responsive grids with no media queries.

NOTES

- › The `fr` unit distributes *free space* — space remaining after fixed tracks are placed
- › `auto-fit` collapses empty tracks; `auto-fill` keeps them — both work with `minmax()`
- › `gap` (formerly `grid-gap`) sets row and column spacing; use `row-gap` / `column-gap` individually
- › Implicit tracks — created when items overflow — are sized by `grid-auto-rows` and `grid-auto-columns`

CODE EXPLAINED

- `display: grid` — container is grid; direct children become grid items
- `repeat(3, 1fr)` — creates 3 equal columns using fractional units
- `fr` — fractional unit: 1 share of remaining free space
- `260px 1fr` — fixed sidebar + fluid main that fills the rest
- `auto-fit` — as many tracks as fit; empty ones collapse to zero
- `minmax(220px, 1fr)` — each track min 220px, max 1fr
- `gap: 1rem` — spacing between all tracks, no edge space
- `grid-template-areas` — named regions for readable item placement

GAP PROPERTIES

SYNTAX	DESCRIPTION
<code>gap: 1rem</code>	Row and column gap equal
<code>gap: 1rem 2rem</code>	Row gap column gap
<code>row-gap: 1rem</code>	Row gap only
<code>column-gap: 2rem</code>	Column gap only

TRACK SIZING VALUES

SYNTAX	DESCRIPTION
<code>fr</code>	Fraction of free space
<code>auto</code>	Size to content
<code>px / rem</code>	Fixed absolute size
<code>%</code>	Percentage of container
<code>minmax(min, max)</code>	Size range
<code>fit-content(N)</code>	Content size capped at N

REPEAT() PATTERNS

SYNTAX	DESCRIPTION
<code>repeat(3, 1fr)</code>	3 equal columns
<code>repeat(4, 200px)</code>	4 fixed columns
<code>repeat(auto-fit, minmax(220px, 1fr))</code>	Responsive columns
<code>repeat(auto-fill, minmax(220px, 1fr))</code>	Same, keeps empty tracks
<code>repeat(3, 1fr 2fr)</code>	Alternating pattern

⚠ COMMON MISTAKES

- › Using `fr` alongside fixed tracks and expecting equal distribution — `fr` only shares the space remaining after fixed tracks
- › Using deprecated `grid-gap` — it still works but the standard is `gap`
- › Confusing `auto-fit` and `auto-fill` — use `auto-fit` for responsive card grids so empty tracks collapse
- › Forgetting `grid-auto-rows` — without it, implicitly created rows are sized to content and can be inconsistent

✓ TIPS & BEST PRACTICES

- › `repeat(auto-fit, minmax(250px, 1fr))` — the one-liner responsive card grid requiring zero media queries
- › Use `grid-template-areas` for page-level layout — it reads like a diagram of your design
- › Set `grid-auto-rows` to a fixed height or `minmax()` for consistent implicit row sizes
- › Combine fixed and `fr` tracks: `260px 1fr` for a layout that adapts without math

QUICK SUMMARY

- › `display: grid` on the container; direct children are grid items
- › `fr` distributes free space · `repeat()` reduces repetition · `minmax()` sets a range
- › `auto-fit` + `minmax()` = fully responsive grid with no media queries
- › `grid-template-areas` gives regions readable names for placement

DESCRIPTION

Grid item properties control where a child is placed within the grid. By default items auto-place into the next available cell. `grid-column` and `grid-row` use 1-based line numbers to place items explicitly. The `span` keyword stretches an item across multiple tracks. Negative line numbers count from the end — `-1` is always the last grid line. `grid-area` can reference a named region from `grid-template-areas` on the container.

EXAMPLE

```
/* Line-number placement */
.item {
  grid-column: 2 / 4;    /* col line 2 → line 4 (2 tracks wide) */
  grid-row:    1 / 3;    /* row line 1 → line 3 (2 rows tall) */
}

/* span keyword */
.wide { grid-column: 1 / span 2; } /* start line 1, span 2 cols */
.auto  { grid-column: span 3; }    /* span 3 from auto position */

/* Full-width item regardless of column count */
.full  { grid-column: 1 / -1; }

/* grid-area shorthand: row-start / col-start / row-end / col-end */
.item  { grid-area: 1 / 1 / 3 / 4; }

/* Named area placement — container must define grid-template-areas */
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.footer  { grid-area: footer; }

/* Alignment within the cell */
.item { justify-self: center; } /* horizontal */
.item { align-self: center; }  /* vertical */
.item { place-self: center; }  /* both at once */
```

NOTES

- Grid lines are numbered from 1; negative numbers count from the end — `-1` is the last line
- `grid-column: 1 / -1` spans the full width regardless of the column count
- `grid-area` shorthand order: row-start / column-start / row-end / column-end
- Auto-placed items follow the grid auto-placement algorithm; explicitly placed items can coexist

CODE EXPLAINED

- `grid-column: 2 / 4` — from column line 2 to line 4 (two columns wide)
- `grid-row: 1 / 3` — from row line 1 to line 3 (two rows tall)
- `span 2` — stretch across 2 tracks from the item's start position
- `1 / -1` — from first to last line = full width of grid
- `grid-area: header` — places item into the named region on the container
- `grid-area: 1 / 1 / 3 / 4` — shorthand: row-start / col-start / row-end / col-end
- `justify-self` — horizontal alignment within the item's cell
- `place-self: center` — centres item both axes within its cell

GRID-COLUMN / GRID-ROW

SYNTAX	DESCRIPTION
<code>2 / 4</code>	From line 2 to line 4
<code>1 / span 3</code>	From line 1, span 3 tracks
<code>span 2</code>	Span 2 from auto position
<code>1 / -1</code>	Full width (first to last line)
<code>2 / -2</code>	All but first and last track

JUSTIFY-SELF / ALIGN-SELF

SYNTAX	DESCRIPTION
<code>start</code>	Align to cell start edge
<code>end</code>	Align to cell end edge
<code>center</code>	Centre within cell
<code>stretch</code>	Fill cell (default)

CONTAINER COUNTERPARTS

SYNTAX	DESCRIPTION
<code>justify-items</code>	Default justify-self for all items
<code>align-items</code>	Default align-self for all items
<code>place-items: center</code>	Centre all items both axes
<code>justify-content</code>	Grid columns in container
<code>align-content</code>	Grid rows in container
<code>place-content</code>	Shorthand: align / justify

COMMON MISTAKES

- Using 0-based line numbers — CSS grid lines start at 1, not 0
- Thinking line numbers equal column count — a 3-column grid has 4 column lines (1 through 4)
- Mixing auto and explicit placement heavily without `grid-auto-flow: dense` — can create visible gaps
- Setting a `grid-area` name when the container has no matching `grid-template-areas`

TIPS & BEST PRACTICES

- `grid-column: 1 / -1` is the safest full-width item — works regardless of how many columns there are
- Use `grid-template-areas` for the page skeleton; use line numbers for fine-grained item tweaks
- Set `grid-auto-flow: dense` on the container to fill gaps when items are different sizes
- `place-self: center` is the quickest way to centre a grid item within its own cell

QUICK SUMMARY

- `grid-column: start / end` — placement using 1-based grid line numbers
- `span N` — stretch across N tracks from the item's start position
- `1 / -1` spans full width; negative numbers count from the end
- `grid-area: name` — place into a named region from `grid-template-areas`

DESCRIPTION

The `position` property controls how an element is placed in or removed from the normal document flow. `static` is the default — elements flow normally. `relative` offsets from its natural position while staying in flow. `absolute` removes the element from flow and anchors it to the nearest positioned ancestor. `fixed` anchors to the viewport; `sticky` behaves as relative until a scroll threshold is reached, then sticks.

NOTES

- › An element is 'positioned' when its `position` is anything other than `static`
- › `absolute` positions relative to the nearest *positioned* ancestor — if none, the initial containing block
- › `z-index` only works on positioned elements (non-static) and flex/grid items
- › A new stacking context is created by: non-static position + z-index, opacity < 1, transform, filter, and others

CODE EXPLAINED

- `position: static` — default; `top/right/bottom/left` ignored
- `position: relative` — stays in flow; offsets from its own natural position
- `position: absolute` — removed from flow; anchored to nearest positioned parent
- `position: relative` on parent — creates the positioning context for absolute children
- `position: fixed` — anchored to the viewport; unaffected by page scrolling
- `position: sticky` — flows normally until the scroll threshold, then sticks
- `inset: 0` — shorthand for `top:0; right:0; bottom:0; left:0`
- `z-index` — higher number = closer to the viewer within the same stacking context

POSITION VALUES	
SYNTAX	DESCRIPTION
<code>static</code>	Default — normal flow, no offsets
<code>relative</code>	Offset from natural position, stays in flow
<code>absolute</code>	Out of flow, relative to positioned ancestor
<code>fixed</code>	Relative to viewport, stays on scroll
<code>sticky</code>	Relative until scroll threshold

OFFSET & INSET	
SYNTAX	DESCRIPTION
<code>top / right / bottom / left</code>	Individual offsets
<code>inset: 0</code>	All four sides = 0
<code>inset: 1rem</code>	All sides equal
<code>inset: 1rem 2rem</code>	Vertical Horizontal
<code>inset: T R B L</code>	Clockwise like padding

COMMON MISTAKES

- › Forgetting `position: relative` on the parent of an absolute child — it anchors to the wrong ancestor
- › Using `z-index` on a `static` element — it has no effect without a non-static position
- › Setting `position: sticky` without a threshold (`top` , `bottom`) — element never sticks
- › Using `overflow: hidden` on a sticky item's parent — it breaks `sticky` behaviour entirely

TIPS & BEST PRACTICES

- › Use `inset: 0` as a shorthand for full-coverage overlays — cleaner than four separate offset properties
- › Maintain a z-index scale in your design tokens: content=1, dropdowns=10, modals=100, toasts=200
- › Use `isolation: isolate` on components to contain their z-index within their own stacking context
- › For sticky table headers, ensure no ancestor has `overflow: hidden` or `overflow: auto`

QUICK SUMMARY

- › `static` (default) · `relative` (in flow, offset) · `absolute` (out of flow)
- › `fixed` = viewport-anchored · `sticky` = relative until scroll threshold
- › Absolute children need a `position: relative` ancestor as their positioning context
- › `z-index` only works on positioned (non-static) elements and flex/grid items

EXAMPLE

```
/* static — default; offset properties have no effect */
.a { position: static; }

/* relative — stays in flow; offset from natural position */
.b { position: relative; top: 10px; left: 20px; }

/* absolute — out of flow; relative to nearest positioned ancestor */
.parent { position: relative; } /* establishes context */
.badge {
  position: absolute;
  top: -8px;
  right: -8px;
}

/* fixed — relative to viewport; stays on scroll */
.nav { position: fixed; top: 0; left: 0; width: 100%; z-index: 100; }

/* sticky — relative until scroll threshold, then fixed */
.thead {
  position: sticky;
  top: 0; /* sticks when it hits top: 0 */
  z-index: 10;
  background: #1c2030;
}

/* inset shorthand — replaces top/right/bottom/left */
.overlay { position: absolute; inset: 0; } /* all sides 0 */
.offset { position: absolute; inset: 1rem 2rem; } /* V | H */

/* z-index — stacking order; higher = in front */
.modal { position: fixed; z-index: 100; }
.overlay { position: fixed; z-index: 99; }
```

STACKING CONTEXT TRIGGERS	
SYNTAX	DESCRIPTION
<code>position + z-index</code>	Classic stacking context
<code>opacity < 1</code>	Any opacity below 1
<code>transform: ...</code>	Any transform value
<code>filter: ...</code>	Any filter value
<code>isolation: isolate</code>	Explicit stacking context
<code>will-change</code>	Some values trigger it

DESCRIPTION

`display` determines how an element participates in layout — as a block, inline, flex container, grid container, or removed entirely. `overflow` controls what happens when content exceeds its container. `visibility: hidden` hides an element while keeping its space in the layout; `display: none` removes it from flow entirely. The legacy `float` is still useful for wrapping text around images but should not be used for page layout.

NOTES

- › `display: none` removes the element from layout — space collapses. `visibility: hidden` hides it but keeps the space
- › `display: contents` makes the element's box disappear while keeping its children in the flow — useful for flex/grid semantics
- › `overflow: auto` is preferred over `scroll` — scrollbars appear only when actually needed
- › Floated elements are pulled out of flow; `overflow: hidden` on the parent or a `clearfix` restores height

CODE EXPLAINED

- `display: block` — fills available width, starts on a new line
- `display: inline` — flows with text; width/height ignored
- `display: inline-block` — inline flow with block sizing
- `display: none` — element and its space are removed from layout
- `display: contents` — the element's box is removed; children stay in flow
- `overflow: hidden` — clips content and clears internal floats
- `overflow: auto` — scrollbar appears only when content overflows
- `visibility: hidden` — invisible but still occupies space

EXAMPLE

```
/* Block vs inline */
.block { display: block; }           /* full width, new line */
.inline { display: inline; }         /* flows with text */
.inline-block { display: inline-block; } /* inline flow + block sizing */
.none { display: none; }             /* removed from layout entirely */
.content { display: contents; }      /* box gone, children remain */

/* Flex and grid */
.flex { display: flex; }
.inline-flex { display: inline-flex; }
.grid { display: grid; }
.inline-grid { display: inline-grid; }

/* Overflow */
.clip { overflow: hidden; }          /* clips — also clears floats */
.scroll { overflow: auto; }          /* scrollbar when needed */
.always { overflow: scroll; }         /* always shows scrollbar */
.axes { overflow-x: auto; overflow-y: hidden; }

/* Visibility vs display */
.hide-space { visibility: hidden; }  /* invisible, space kept */
.hide-flow { display: none; }        /* invisible, space removed */
.see-thru { opacity: 0; }            /* invisible, space kept, clickable */

/* Float — text wrapping only */
.figure { float: left; margin: 0 1rem 1rem 0; }
.clear { clear: both; }

/* Modern clearfix */
.clearfix::after { content: ''; display: block; clear: both; }
```

DISPLAY VALUES			
SYNTAX	DESCRIPTION	SYNTAX	DESCRIPTION
<code>block</code>	Full width, new line	<code>grid</code>	Block-level grid container
<code>inline</code>	Flows with text	<code>inline-grid</code>	Inline-level grid container
<code>inline-block</code>	Inline + block sizing	<code>none</code>	Removed from layout
<code>flex</code>	Block-level flex container	<code>contents</code>	Box removed, children remain
<code>inline-flex</code>	Inline-level flex container		

OVERFLOW VALUES	
SYNTAX	DESCRIPTION
<code>visible</code>	Default — spills outside
<code>hidden</code>	Clips overflowing content
<code>scroll</code>	Always shows scrollbars
<code>auto</code>	Scrollbar only when needed
<code>clip</code>	Like hidden, no scroll API

VISIBILITY COMPARISON	
SYNTAX	DESCRIPTION
<code>visibility: visible</code>	Shown (default)
<code>visibility: hidden</code>	Hidden, space kept
<code>display: none</code>	Hidden, space gone
<code>opacity: 0</code>	Invisible, space kept, clickable
<code>pointer-events: none</code>	Visible but not interactive

COMMON MISTAKES

- › Using `visibility: hidden` when you need `display: none` — the element still occupies space
- › Using `overflow: hidden` on a sticky element's parent — it breaks `position: sticky`
- › Floating elements without clearing — the parent collapses to zero height
- › Using `display: none` to hide content meant for screen readers — use the `.sr-only` pattern instead

TIPS & BEST PRACTICES

- › Use flex or grid for all new layouts — floats are only for text wrapping around images
- › The `.sr-only` pattern (`position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0,0,0,0)`) hides visually but keeps content for screen readers
- › `overflow: hidden` on a parent is a quick float-clearing trick — prefer flex/grid to avoid needing it
- › `display: contents` is great for making a semantically required wrapper invisible to flex/grid layout

QUICK SUMMARY

- › `block` = full width · `inline` = text flow · `inline-block` = both
- › `display: none` removes from flow · `visibility: hidden` hides but keeps space
- › `overflow: auto` for scrollable containers · `hidden` to clip or clear floats
- › Use flex or grid for layouts — floats are for text wrapping only

DESCRIPTION

CSS transitions animate a property change smoothly from one value to another when a state change occurs — such as `:hover`, `:focus`, or a class toggle via JavaScript. You define *which* property to animate, *how long* it takes, the *easing* curve, and an optional *delay*. Transitions are reactive — they only fire when a triggering change happens, unlike animations which run independently.

NOTES

- Only *animatable* properties can transition — most numeric values qualify; `display` cannot
- `transition: all` is convenient but animates every changing property — can cause unexpected results and hurt performance
- Place `transition` on the *base state*, not the `:hover` — so it plays in both directions (on and off)
- GPU-accelerated: `opacity` and `transform` — always prefer these for smooth 60 fps animations

CODE EXPLAINED

- `transition: background 0.2s ease` — property, duration, timing
- Comma-separated list — each property gets its own duration and timing
- `transition: all` — transitions every animatable property (use carefully)
- `ease` — slow start, fast middle, slow end — natural feel (default)
- `ease-in` — starts slowly, accelerates into the end
- `ease-out` — fast start, decelerates — best for elements entering screen
- `cubic-bezier()` — fully custom easing with 4 control points
- Transition on the base state — it then runs in both directions automatically

EXAMPLE

```
/* Shorthand: property | duration | timing-function | delay */
.btn { transition: background-color 0.2s ease; }
.btn { transition: all 0.3s ease-in-out; }

/* Multiple properties — comma separated */
.card {
  transition:
    transform 0.3s ease,
    box-shadow 0.3s ease,
    opacity 0.2s ease 0.05s; /* delay on opacity */
}

/* Timing functions */
.a { transition-timing-function: ease; } /* default */
.b { transition-timing-function: linear; }
.c { transition-timing-function: ease-in; } /* slow start */
.d { transition-timing-function: ease-out; } /* slow end */
.e { transition-timing-function: ease-in-out; } /* slow both */
.f { transition-timing-function: cubic-bezier(0.34, 1.56, 0.64, 1); }

/* Correct pattern — transition on BASE state */
.btn {
  background: #5c6bc0;
  transition: background 0.2s ease, transform 0.2s ease;
}
.btn:hover {
  background: #7986cb;
  transform: translateY(-2px);
}
```

TIMING FUNCTIONS

SYNTAX	DESCRIPTION
<code>ease</code>	Slow-fast-slow (default)
<code>linear</code>	Constant speed
<code>ease-in</code>	Slow start, fast end
<code>ease-out</code>	Fast start, slow end
<code>ease-in-out</code>	Slow start and end
<code>steps(N)</code>	N discrete jumps
<code>cubic-bezier()</code>	Custom bezier curve

SHORTHAND PARTS

SYNTAX	DESCRIPTION
<code>property</code>	CSS property name or all
<code>duration</code>	Time in s or ms
<code>timing-function</code>	Easing keyword or function
<code>delay</code>	Wait before starting (default 0s)

GPU-FRIENDLY PROPERTIES

SYNTAX	DESCRIPTION
<code>opacity</code>	Fade — composited layer
<code>transform</code>	Move/scale/rotate — no layout recalc
<code>filter</code>	Blur/brightness — GPU composited
<code>color</code>	Text colour — paint only
<code>background-color</code>	Background — paint only
<code>width / height</code>	⚠ Triggers layout — avoid if possible

⚠ COMMON MISTAKES

- Putting `transition` on the `:hover` state only — the reverse transition back won't animate
- Using `transition: all` in production — animates properties you didn't intend to animate
- Transitioning `width` or `height` — triggers layout recalc; use `transform: scaleX()` instead
- Expecting `display: none` to transition — it is not an animatable property

✓ TIPS & BEST PRACTICES

- Stick to `opacity` and `transform` for performance-critical transitions — they run on the GPU
- Keep UI transition durations short: 150–250ms feels instant; 300–400ms feels polished; longer feels sluggish
- Use incremented `transition-delay` on list items for a staggered reveal effect
- `cubic-bezier(0.34, 1.56, 0.64, 1)` creates a satisfying spring/overshoot bounce on hover

QUICK SUMMARY

- Shorthand: `property duration timing-function delay`
- Place `transition` on the base state — it animates in both directions automatically
- GPU-accelerated: `opacity` and `transform` — prefer over layout-triggering properties
- Multiple transitions comma-separated — each can have independent timing

DESCRIPTION

CSS animations use `@keyframes` rules to define intermediate steps and run independently without a trigger. Unlike transitions, they can loop, play automatically on page load, and define multiple intermediate steps. The `animation` shorthand sets name, duration, timing, delay, iteration count, direction, fill-mode, and play-state in one declaration.

NOTES

- › Keyframe percentages: `0%` = start, `100%` = end — the keywords `from` and `to` are equivalent
- › `animation-fill-mode: forwards` — element keeps the final keyframe styles after the animation ends
- › `animation-direction: alternate` — reverses on each even iteration creating a ping-pong effect
- › Respect user preferences: wrap non-essential animations in `@media (prefers-reduced-motion: no-preference)`

CODE EXPLAINED

- `@keyframes fadeIn` — defines named animation steps referenced by `animation-name`
- `from / to` — shorthand for `0% / 100%`
- `0% 50% 100%` — define as many intermediate steps as needed
- `animation: fadeIn 0.4s ease-out forwards` — name duration timing fill-mode
- `infinite` — loops the animation forever
- `forwards` fill-mode — element retains the final keyframe state after finishing
- `both` fill-mode — applies first frame before delay, retains last frame after end
- `animation-play-state: paused` — freezes the animation in place

ANIMATION SHORTHAND ORDER

SYNTAX	DESCRIPTION
<code>animation-name</code>	Name matching <code>@keyframes</code>
<code>animation-duration</code>	Length of one cycle
<code>animation-timing-function</code>	Easing keyword or function
<code>animation-delay</code>	Wait before first run
<code>animation-iteration-count</code>	Number or infinite
<code>animation-direction</code>	normal reverse alternate
<code>animation-fill-mode</code>	none forwards backwards both
<code>animation-play-state</code>	running paused

COMMON MISTAKES

- › Forgetting `animation-fill-mode: forwards` — element snaps back to original state after ending
- › Animating layout-triggering properties (`width`, `margin`) — use `transform` for 60 fps
- › Defining `@keyframes` inside a selector — keyframes must be at the top level of the stylesheet
- › Not wrapping decorative animations in `prefers-reduced-motion` — causes issues for users with vestibular disorders

EXAMPLE

```
/* Define keyframes */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(10px); }
  to   { opacity: 1; transform: translateY(0); }
}

@keyframes pulse {
  0%   { transform: scale(1); }
  50%  { transform: scale(1.05); }
  100% { transform: scale(1); }
}

@keyframes spin {
  to { transform: rotate(360deg); }
}

/* Apply: name | duration | timing | delay | count | direction | fill */
.card { animation: fadeIn 0.4s ease-out forwards; }
.spinner { animation: spin 1s linear infinite; }
.badge { animation: pulse 2s ease-in-out infinite; }

/* fill-mode: both — apply from: before delay, keep to: after end */
.item { animation: fadeIn 0.4s ease-out 0.1s both; }

/* Staggered entrance */
.item:nth-child(1) { animation: fadeIn 0.4s ease-out 0.0s both; }
.item:nth-child(2) { animation: fadeIn 0.4s ease-out 0.1s both; }
.item:nth-child(3) { animation: fadeIn 0.4s ease-out 0.2s both; }

/* Pause on hover */
:animated:hover { animation-play-state: paused; }

/* Reduced motion */
@media (prefers-reduced-motion: reduce) {
  * { animation-duration: 0.01ms !important; }
}
```

FILL-MODE VALUES

SYNTAX	DESCRIPTION
<code>none</code>	Default — snaps back after end
<code>forwards</code>	Keeps final keyframe state
<code>backwards</code>	Applies first keyframe before delay
<code>both</code>	Combines forwards and backwards

DIRECTION VALUES

SYNTAX	DESCRIPTION
<code>normal</code> (default)	Forward each iteration
<code>reverse</code>	Backward each iteration
<code>alternate</code>	Forward then backward
<code>alternate-reverse</code>	Backward then forward

TIPS & BEST PRACTICES

- › Use `animation-fill-mode: both` to apply the first frame before a delay and keep the last after
- › Stagger `animation-delay` on list children for a polished cascade entrance effect
- › Animate only `opacity` and `transform` for GPU-accelerated 60 fps performance
- › Name keyframes descriptively: `slideInFromLeft`, `fadeUp`, `pulseShadow`

QUICK SUMMARY

- › `@keyframes name { from {} to {} }` — defines the animation steps
- › Shorthand: `name | duration | timing | delay | count | direction | fill-mode`
- › `forwards` = keeps final state · `both` = forwards + backwards · `infinite` = loops
- › Animate `opacity` and `transform` only for GPU-accelerated performance

DESCRIPTION

The `transform` property applies geometric operations to elements — moving, rotating, scaling, or skewing — without affecting document layout flow. Transformed elements don't push neighbouring elements. Multiple functions are chained in one `transform` declaration and applied **right-to-left**. Because transforms don't trigger layout recalculation, they are the preferred technique for animating motion at 60 fps.

- NOTES**
- Transforms are applied right-to-left: `rotate(45deg) translateX(100px)` rotates first, then translates in the rotated direction
 - `transform-origin` defaults to the element centre (50% 50%) — change it to adjust the pivot point
 - Applying any `transform` creates a new stacking context — the element paints above non-transformed siblings
 - Modern CSS has standalone `translate`, `rotate`, and `scale` properties — they can be animated independently

- CODE EXPLAINED**
- `translate(X, Y)` — moves element; doesn't affect surrounding layout
 - `translate(50%)` — % is relative to the element's own dimensions, not the parent
 - `rotate(45deg)` — clockwise; also accepts `turn`, `rad`, `grad`
 - `scale(1.2)` — 120% of original size; values below 1 shrink
 - `skew(Xdeg, Ydeg)` — slants along X and/or Y axes
 - Chain order matters — functions apply right to left
 - `transform-origin` — pivot point for rotate and scale (default: 50% 50%)
 - `perspective` on the parent — enables visible depth for 3D child transforms

TRANSFORM-ORIGIN VALUES	
SYNTAX	DESCRIPTION
50% 50%	Centre (default)
top left	Top-left corner
0 0	Same as top left
bottom center	Bottom centre
50% 100%	Bottom centre (numeric)
20px 30px	Absolute offset

TRANSFORM FUNCTIONS			
SYNTAX	DESCRIPTION	SYNTAX	DESCRIPTION
<code>translate(x, y)</code>	Move on X and Y	<code>rotateX/Y/Z(deg)</code>	3D rotation per axis
<code>translateX(n)</code>	Move horizontally	<code>scale(x, y)</code>	Scale both axes
<code>translateY(n)</code>	Move vertically	<code>scaleX(n) / scaleY(n)</code>	Scale one axis
<code>translateZ(n)</code>	Move in Z (needs perspective)	<code>skew(x, y)</code>	Slant on X and Y
<code>rotate(deg)</code>	2D rotation	<code>matrix(a,b,c,d,e,f)</code>	Full 2D matrix

- COMMON MISTAKES**
- `translate(50%)` is 50% of the element itself, not the parent — easy to confuse
 - Ignoring chain order — `rotate then translate` is completely different from `translate then rotate`
 - Using `top/left` for animations instead of `transform: translate()` — `top/left` trigger layout recal
 - Forgetting `perspective` on the parent for 3D — without it 3D transforms show no depth

- TIPS & BEST PRACTICES**
- Always animate with `transform` and `opacity` — the only properties that don't trigger layout or paint
 - `transform: translate(-50%, -50%)` with absolute positioning is the classic true-centring trick
 - Set `transform-origin: top center` for dropdown menus that should grow downward from their trigger
 - Use standalone `translate`, `rotate`, `scale` properties in modern CSS for independent animation

EXAMPLE

```
/* Translate — move without affecting layout */
.a { transform: translateX(50px); }
.b { transform: translateY(-20px); }
.c { transform: translate(50px, -20px); }
.d { transform: translate(50%); } /* % of element's own size */

/* Rotate */
.e { transform: rotate(45deg); }
.f { transform: rotate(-0.25turn); } /* turns unit */

/* Scale */
.g { transform: scale(1.2); } /* uniform 120% */
.h { transform: scaleX(2); } /* horizontal only */
.i { transform: scale(1.2, 0.8); } /* x and y independently */

/* Skew */
.j { transform: skewX(15deg); }
.k { transform: skew(15deg, 5deg); }

/* Chaining — applied right to left */
.l { transform: rotate(45deg) scale(1.2) translateX(20px); }

/* Transform origin */
.m { transform-origin: top left; }
.n { transform-origin: 50% 100%; } /* bottom centre */

/* 3D */
.wrap { perspective: 600px; } /* set on parent */
.box { transform: rotateY(45deg); }
.box { transform: translateZ(60px); }
```

ANGLE UNITS	
SYNTAX	DESCRIPTION
deg	Degrees — 360 = full circle
turn	Turns — 1turn = 360deg
rad	Radians — 2π = 360deg
grad	Gradians — 400 = 360deg

- QUICK SUMMARY**
- `transform`: translate · rotate · scale · skew — no layout impact
 - Functions apply right-to-left — order significantly changes the visual result
 - `transform-origin` sets the pivot; default is element centre (50% 50%)
 - Always prefer `transform` over `top/left` for animation — GPU-accelerated

EXAMPLE

```
/* Mobile-first: base styles apply to all screen sizes */
.container { padding: 1rem; }

@media (min-width: 640px) { /* sm: tablet portrait and up */
  .container { padding: 1.5rem; }
}

@media (min-width: 768px) { /* md: tablet landscape and up */
  .container { padding: 2rem; }
  .grid { display: grid; grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 1024px) { /* lg: desktop and up */
  .grid { grid-template-columns: repeat(3, 1fr); }
}

@media (min-width: 1280px) { /* xl: wide desktop */
  .container { max-width: 1200px; margin: 0 auto; }
}

/* Preference queries */
@media (prefers-color-scheme: dark) { body { background: #111; color: #eee; } }
@media (prefers-reduced-motion: reduce) { * { animation: none !important; } }
@media (hover: none) { .tooltip { display: none; } }

/* Print */
@media print {
  .nav, .footer, .sidebar { display: none; }
  body { font-size: 12pt; color: black; background: white; }
  ::after { content: ' (' attr(href) ')'; }
}

/* Range syntax (modern) */
@media (640px <= width < 1024px) { /* between sm and lg */ }
```

DESCRIPTION

Media queries apply CSS conditionally based on device or environment characteristics. Modern responsive design uses a **mobile-first** approach: base styles target small screens, and `min-width` queries progressively enhance for larger ones. This requires fewer overrides than desktop-first and aligns with how browsers on slow connections render pages. Beyond width, media queries can target orientation, colour scheme preference, motion preference, and print.

NOTES

- Mobile-first uses `min-width` — styles apply from that breakpoint *upward*
- Desktop-first uses `max-width` — styles apply from that breakpoint *downward* — requires more overrides
- Choose breakpoints based on your content, not specific devices — devices change; content doesn't
- Modern range syntax (`640px <= width < 1024px`) replaces chained min/max queries

CODE EXPLAINED

- Base styles — no `@media` wrapper means they apply to all sizes
- `min-width: 640px` — activates for viewports 640px wide and above
- Stack queries smallest → largest for mobile-first; no need to undo previous rules
- `prefers-color-scheme: dark` — system dark mode is active
- `prefers-reduced-motion: reduce` — user requests minimal animation
- `hover: none` — device has no true hover capability (touch)
- `@media print` — applied only in print preview and when printing
- Range syntax: `640px <= width < 1024px` — cleaner than chained min/max

COMMON BREAKPOINTS

SYNTAX	DESCRIPTION
<code>min-width: 480px</code>	xs — large phones
<code>min-width: 640px</code>	sm — tablet portrait
<code>min-width: 768px</code>	md — tablet landscape
<code>min-width: 1024px</code>	lg — desktop
<code>min-width: 1280px</code>	xl — wide desktop
<code>min-width: 1536px</code>	2xl — ultra-wide

MEDIA FEATURES

SYNTAX	DESCRIPTION
<code>width / height</code>	Viewport dimensions
<code>orientation</code>	portrait or landscape
<code>prefers-color-scheme</code>	light or dark
<code>prefers-reduced-motion</code>	reduce or no-preference
<code>hover</code>	hover or none
<code>pointer</code>	fine (mouse) or coarse (touch)
<code>resolution</code>	Device pixel density

MEDIA TYPES

SYNTAX	DESCRIPTION
<code>all</code>	All devices (default)
<code>screen</code>	Screens and displays
<code>print</code>	Print preview and printer
<code>speech</code>	Screen readers

COMMON MISTAKES

- Using desktop-first `max-width` — leads to more override rules and greater complexity as screens shrink
- Picking breakpoints to match specific devices — designs break as device sizes shift over time
- Having no print stylesheet — pages can print with dark backgrounds, nav bars, and illegible colours
- Animating without `prefers-reduced-motion` — causes nausea and dizziness for users with vestibular disorders

✓ TIPS & BEST PRACTICES

- Write mobile-first: add complexity for larger screens, never strip it away for smaller ones
- Use `@media (hover: hover)` to restrict hover effects to devices that genuinely support them
- Wrap all non-essential animation declarations in `prefers-reduced-motion: no-preference`
- Always include a print stylesheet: hide navigation, use black text, reveal link URLs via `attr(href)`

QUICK SUMMARY

- Mobile-first: base styles for small screens, `min-width` queries enhance upward
- Common breakpoints: 640 / 768 / 1024 / 1280px — base on content, not device names
- `prefers-color-scheme` and `prefers-reduced-motion` respect OS-level user settings
- `@media print` — hide nav, use black text, remove backgrounds for clean printing

DESCRIPTION

Fluid units and CSS math functions let layouts and typography adapt to the viewport without breakpoints. `rem` scales with the root font size; `em` scales with the local font size. Viewport units (`vw`, `vh`) are relative to the screen. `clamp(min, preferred, max)` creates self-constraining fluid values — a single declaration that replaces a minimum, maximum, and in-between media query. `calc()` mixes units at render time.

NOTES

- › `1rem` equals the root font-size (typically 16px) — it respects the user's browser font-size setting
- › `em` is relative to the element's own font-size for spacing; nesting compounds the value
- › `100svh` (small viewport height) avoids the mobile browser chrome problem with `100vh`
- › Spaces around `+` and `-` in `calc()` are required — `calc(100%-2rem)` is invalid

CODE EXPLAINED

- `rem` — root-relative; predictable and respects browser font-size settings
- `em` — parent-relative for font-size; self-relative for padding/margin
- `100vh` — full viewport height; can include mobile browser UI
- `100svh` — small viewport: excludes mobile address bar and tab strip
- `calc(100% - 260px)` — mixes percentage and px; resolved at paint time
- `clamp(min, preferred, max)` — the preferred scales fluidly, clamped by min/max
- `5vw` as the preferred in clamp — scales with viewport width
- `min(100%, 600px)` — always picks the smaller value; equivalent to `max-width`

CLAMP() PATTERNS

SYNTAX	DESCRIPTION
<code>clamp(1rem, 4vw, 2rem)</code>	Fluid body text
<code>clamp(2rem, 6vw, 4rem)</code>	Fluid heading
<code>clamp(320px, 90%, 1200px)</code>	Fluid container width
<code>clamp(1rem, 2vw + 0.5rem, 2rem)</code>	Fluid spacing

UNIT REFERENCE

SYNTAX	DESCRIPTION	SYNTAX	DESCRIPTION
<code>px</code>	Absolute pixels	<code>vw / vh</code>	1% of viewport width/height
<code>rem</code>	Root font-size relative	<code>svw / svh</code>	Small viewport (excl. chrome)
<code>em</code>	Local font-size relative	<code>dvw / dvh</code>	Dynamic viewport (live update)
<code>%</code>	Relative to parent value	<code>cqw / cqh</code>	Container query width/height

COMMON MISTAKES

- › Using `em` for font-size in nested components — values compound and become unpredictable
- › Using `100vh` for full-screen mobile layouts — the address bar is included and content gets cut off
- › Omitting spaces around `+` and `-` in `calc()` — the expression fails silently
- › Using bare `vw` for font-size without `clamp()` — text is too small on phones, too large on wide screens

EXAMPLE

```
/* rem — relative to :root font-size */
:root { font-size: 100%; } /* respects browser setting */
h1 { font-size: 2.5rem; } /* 40px if root = 16px */
p { font-size: 1rem; } /* 16px */

/* em — relative to element's own font-size */
.parent { font-size: 20px; }
.child { font-size: 0.8em; } /* 16px — 80% of parent */
.child { padding: 1em; } /* 16px — relative to child's own size */

/* Viewport units */
.hero { height: 100vh; } /* full viewport height */
.hero { height: 100svh; } /* safe: excludes mobile chrome */
.banner { width: 50vw; }

/* calc() — mix any units */
.sidebar { width: calc(100% - 260px); }
.pad { padding: calc(1rem + 2vw); }

/* clamp(minimum, preferred, maximum) */
h1 { font-size: clamp(1.5rem, 5vw, 3rem); }
.container { width: clamp(320px, 90%, 1200px); }
.gap { gap: clamp(1rem, 3vw, 2rem); }

/* min() and max() */
.box { width: min(100%, 600px); } /* never wider than 600px */
.box { width: max(300px, 50%); } /* never narrower than 300px */
```

MATH FUNCTIONS

SYNTAX	DESCRIPTION	SYNTAX	DESCRIPTION
<code>calc(a + b)</code>	Add — spaces required around <code>+</code>	<code>min(a, b)</code>	Picks the smaller value
<code>calc(a - b)</code>	Subtract	<code>max(a, b)</code>	Picks the larger value
<code>calc(a * b)</code>	Multiply — one must be unitless	<code>clamp(min, val, max)</code>	Constrained preferred value
<code>calc(a / b)</code>	Divide — divisor must be unitless		

TIPS & BEST PRACTICES

- › Use `clamp()` for fluid typography and containers — replaces 2–3 media query breakpoints in one line
- › Set `font-size: 100%` on `:root` to preserve the user's browser font-size as the `rem` base
- › `min(100%, 1200px)` is equivalent to `max-width: 1200px; width: 100%` — shorter and cleaner
- › Combine `font-size: clamp(1rem, 2.5vw, 1.5rem)` for fluid scaling with hard safety bounds

QUICK SUMMARY

- › `rem` = root-relative (predictable) · `em` = local-relative (compounds)
- › `vh/vw` = viewport · use `svh` for mobile safe full-height
- › `clamp(min, preferred, max)` — one line replaces multiple breakpoints
- › `calc()` mixes units; spaces required around `+` and `-` operators

DESCRIPTION

CSS custom properties — variables — store reusable values that can be referenced anywhere with `var()`. Declared with a `--` prefix, they cascade and inherit like any CSS property. A variable on `:root` is globally available; one on a component is scoped to that subtree. Unlike Sass variables, CSS custom properties are *live* — they respond to media queries, can be overridden in component scope, and can be read and written by JavaScript.

NOTES

- Custom property names are case-sensitive: `--Color` and `--color` are different variables
- `var()` accepts a fallback: `var(--color, red)` — used when the variable is undefined or invalid
- Custom properties can store any CSS value including space-separated values, strings, and calc expressions
- JS reads/writes them: `el.style.setProperty('--x', val)` and `getComputedStyle(el).getPropertyValue('--x')`

CODE EXPLAINED

- `--name: value` — declares a custom property on the selected element
- `var(--name)` — inserts the variable's value at that point
- `var(--name, fallback)` — fallback used when variable is undefined
- `:root` — the `<html>` element; variables here are globally available
- Component scope: redefining on `.modal` affects only `.modal` and its children
- `[data-theme='dark']` — toggle entire colour themes by flipping one attribute
- `@media + :root` — update all spacing/sizing globally at one breakpoint
- JS `setProperty` — live update without re-parsing any stylesheet

JS INTERACTION

SYNTAX	DESCRIPTION
<code>getComputedStyle(el).getPropertyValue('--x')</code>	Read variable
<code>el.style.setProperty('--x', val)</code>	Write variable
<code>el.style.removeProperty('--x')</code>	Remove override

DECLARATION & USAGE

SYNTAX	DESCRIPTION
<code>--name: value</code>	Declare a custom property
<code>var(--name)</code>	Use the value
<code>var(--name, fallback)</code>	Use value or fallback
<code>:root { --x: val }</code>	Global scope
<code>.component { --x: val }</code>	Scoped override

⚠ COMMON MISTAKES

- Declaring variables without `:root` and expecting global access — scope is limited to descendants of the selector
- Forgetting case-sensitivity — `--Color` is not the same as `--color`
- Assuming custom properties compute eagerly — they store literal text; the value is resolved at use time
- Not providing fallbacks for critical UI values — a failed variable resolves to the property's initial value

EXAMPLE

```
/* Declare on :root for global scope */
:root {
  --color-primary:    hsl(231 44% 56%);
  --color-danger:     hsl(0 70% 60%);
  --color-surface:    #1c2030;
  --spacing-sm:       0.5rem;
  --spacing-md:       1rem;
  --spacing-lg:       2rem;
  --radius-card:      8px;
  --font-body:        'DM Sans', sans-serif;
  --transition-base:  0.2s ease;
}

/* Use with var() */
.btn {
  background:    var(--color-primary);
  padding:      var(--spacing-sm) var(--spacing-md);
  border-radius: var(--radius-card);
  transition:    background var(--transition-base);
}

/* Fallback value — used if variable is undefined */
.box { color: var(--color-accent, #9c8fff); }

/* Component-scoped override */
.modal { --radius-card: 12px; } /* only inside .modal */

/* Theme switching via data attribute */
:root { --bg: #ffffff; --text: #111111; }
[data-theme="dark"] { --bg: #0f1117; --text: #e8eaf6; }
body { background: var(--bg); color: var(--text); }

/* Responsive — update tokens at a breakpoint */
:root { --spacing-lg: 1.5rem; }
@media (min-width: 768px) { :root { --spacing-lg: 2rem; } }
```

TOKEN NAMING PATTERNS

SYNTAX	DESCRIPTION
<code>--color-*</code>	<code>--color-primary</code> , <code>--color-danger</code>
<code>--spacing-*</code>	<code>--spacing-sm</code> , <code>--spacing-xl</code>
<code>--font-*</code>	<code>--font-body</code> , <code>--font-mono</code>
<code>--radius-*</code>	<code>--radius-sm</code> , <code>--radius-full</code>
<code>--shadow-*</code>	<code>--shadow-card</code> , <code>--shadow-modal</code>
<code>--transition-*</code>	<code>--transition-base</code> , <code>--transition-slow</code>

✓ TIPS & BEST PRACTICES

- Keep all custom property declarations in a dedicated `_tokens.css` file — the single source of truth
- Combine with `@media` on `:root` to update spacing, type size, and layout values globally at one breakpoint
- Use `[data-theme]` attribute toggling for dark/light themes — swap every colour with one JS attribute change
- Custom properties work inside `calc()`: `calc(var(--spacing-md) * 2)`

QUICK SUMMARY

- Declare: `--name: value` · Use: `var(--name)` · Fallback: `var(--name, default)`
- `:root` scope = global · component scope = descendants only
- Unlike Sass, CSS custom properties are live — they update with media queries and JS in real time
- Use a `--category-variant` token pattern for a maintainable, scalable design system

DESCRIPTION

Modern CSS features dramatically reduce the need for workarounds. `:is()` and `:where()` group selectors without repetition. `:has()` — the long-awaited parent selector — styles an element based on what it contains. `@layer` gives explicit, predictable control over cascade order. `@container` queries style elements based on their parent's size rather than the viewport. `subgrid` lets nested grids align to the parent grid's tracks.

NOTES

- `:is()` inherits the specificity of its most specific argument; `:where()` always has zero specificity
- `:has()` is supported in all major browsers since late 2023 — it is no longer experimental
- `@layer` order is set by the first declaration — subsequent rules of the same layer name append to it
- `@container` requires `container-type: inline-size` (or `size`) on the parent element

CODE EXPLAINED

- `:is(h1, h2, h3)` — matches all listed selectors; reduces repetitive rule blocks
- `:where()` — same matching as `:is()` but contributes zero specificity
- `.card:has(img)` — selects `.card` elements that contain an `img`
- `:has(:invalid)` — form that contains an invalid input field
- `@layer base, components, utilities` — later layers beat earlier ones at equal specificity
- `@layer utilities { }` — rules here always beat `components`
- `container-type: inline-size` — makes element a named container query context
- `grid-template-columns: subgrid` — child inherits parent grid's column definitions

SELECTOR FUNCTIONS

SYNTAX	DESCRIPTION
<code>:is(a, b, c)</code>	Match any; specificity = highest arg
<code>:where(a, b, c)</code>	Match any; always 0 specificity
<code>:has(selector)</code>	Parent contains matching child
<code>:not(selector)</code>	Excludes matching elements
<code>:is() :where()</code>	Can be nested and combined

@LAYER BEHAVIOUR

SYNTAX	DESCRIPTION
<code>@layer base, components</code>	Declares and orders layers
<code>@layer name { }</code>	Assigns rules to a layer
<code>@import url() layer()</code>	Import file into a layer
Unlayered rules	Always beat layered rules
Later layers	Beat earlier at same specificity

⚠ COMMON MISTAKES

- Confusing `:is()` and `:where()` — use `:where()` for resets (zero specificity is always overridable)
- Using `@container` without `container-type` on the parent — queries never fire
- Assuming `@layer` overrides `!important` — `!important` in a lower layer still beats a higher one
- Using complex `:has()` selectors on very large DOMs without performance testing

✓ TIPS & BEST PRACTICES

- Replace long repeated selector groups with `:is()`:
`:is(h1,h2,h3,h4,h5,h6) { }`
- Use `:where()` in design systems and resets — zero specificity means any consumer selector can override
- `:has()` enables true component-level state styling:
`.card:has(.badge), nav:has(.active)`
- Combine `@layer` + `@container` in new projects for a fully modern, maintainable architecture

QUICK SUMMARY

- `:is()` groups selectors (inherits specificity) · `:where()` groups with zero specificity
- `:has()` — select a parent by what it contains; broadly supported since 2023
- `@layer` — explicit cascade ordering; unlayered rules always beat layered ones
- `@container` — style components based on their parent size, not the viewport

EXAMPLE

```
/* :is() — groups selectors; specificity = highest arg */
:is(h1, h2, h3) { line-height: 1.2; }
:is(article, section, aside) p { margin-bottom: 1rem; }

/* :where() — same as :is() but always zero specificity */
:where(h1, h2, h3, h4, h5, h6) { font-weight: 700; }

/* :has() — parent/relational selector */
.card:has(img)           { padding: 0; }           /* card containing an image */
.form:has(:invalid)      { border-color: red; } /* form with invalid input */
li:has(+ li)             { border-bottom: 1px solid #333; }
nav:has(.active)         { background: #1c2030; }

/* @layer — explicit cascade ordering */
@layer base, components, utilities;

@layer base      { body { margin: 0; font-family: sans-serif; } }
@layer components { .btn { padding: 0.5rem 1rem; border-radius: 4px; } }
@layer utilities { .mt-4 { margin-top: 1rem; } }

/* @container — style based on parent container size */
.card-wrapper {
  container-type: inline-size;
  container-name: card;
}
@container card (min-width: 400px) {
  .card { display: flex; gap: 1rem; }
}

/* subgrid — inherit parent grid tracks */
.parent { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; }
.child {
  grid-column: span 3;
  display: grid;
  grid-template-columns: subgrid; /* aligns columns to parent tracks */
}
```

@CONTAINER SETUP

SYNTAX	DESCRIPTION
<code>container-type: inline-size</code>	Query by width (most common)
<code>container-type: size</code>	Query by width and height
<code>container-name: name</code>	Named container for targeting
<code>@container (min-width: N)</code>	Query — same syntax as <code>@media</code>
<code>grid-template-columns: subgrid</code>	Inherit parent grid tracks